

Formal Verification In Hardware Design: A Survey

CHRISTOPH KERN and MARK R. GREENSTREET

University of British Columbia

In recent years, formal methods have emerged as an alternative approach to ensuring the quality and correctness of hardware designs, overcoming some of the limitations of traditional validation techniques such as simulation and testing.

There are two main aspects to the application of formal methods in a design process: the formal framework used to specify desired properties of a design and the verification techniques and tools used to reason about the relationship between a specification and a corresponding implementation. We survey a variety of frameworks and techniques proposed in the literature and applied to actual designs. The specification frameworks we describe include temporal logics, predicate logic, abstraction and refinement, as well as containment between ω -regular languages. The verification techniques presented include model checking, automata-theoretic techniques, automated theorem proving, and approaches that integrate the above methods.

In order to provide insight into the scope and limitations of currently available techniques, we present a selection of case studies where formal methods were applied to industrial-scale designs, such as microprocessors, floating-point hardware, protocols, memory subsystems, and communications hardware.

Categories and Subject Descriptors: A.1 [**General Literature**]: Introductory and Survey; B.7.2 [**Integrated Circuits**]: Design Aids—*Verification*

General Terms: Design, Verification

Additional Key Words and Phrases: Case studies, formal methods, formal verification, hardware verification, language containment, model checking, survey, theorem proving

This work was supported in part by NSERC research grant OGP-0138501, a UBC graduate fellowship, and a BC Advanced Systems Institute faculty fellowship

Authors' addresses: C. Kern, Department of Computer Science, University of British Columbia, 2366 Main Mall, Vancouver, B.C. V6T 1Z4, Canada; email: ckern@cs.ubc.ca; M. R. Greenstreet, Department of Computer Science, University of British Columbia, 2366 Main Mall, Vancouver, B.C. V6T 1Z4, Canada ; email: mrg@cs.ubc.ca.

Permission to make digital/hard copy of part or all of this work for personal or classroom use is granted without fee provided that the copies are not made or distributed for profit or commercial advantage, the copyright notice, the title of the publication, and its date appear, and notice is given that copying is by permission of the ACM, Inc. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee.

© 1999 ACM 1084-4309/99/0400-0123 \$5.00

ACM Transactions on Design Automation of Electronic Systems, Vol. 4, No. 2, April 1999, Pages 123–193.

1. INTRODUCTION

The correct design of complex hardware poses serious challenges. Economic pressures in rapidly evolving markets demand short design cycles while increasing complexity of designs makes simulation coverage less and less complete. Bugs in a design that are not uncovered in early design stages can be costly, and bugs that remain undetected until after a product is shipped can be extremely expensive. In safety-critical applications, such as nuclear power control or aviation, correct operation of a device is imperative.

Formal methods, i.e., the application of mathematical methodologies to the specification and validation of systems, can aid in tackling these challenges. This survey attempts to provide an understanding of what can be specified, how these properties can be verified, and presents examples of how formal methods have been applied to realistic designs. Earlier surveys include Gupta [1992]; McFarland [1993]; and Shankar [1993]. The Gupta survey, in particular, provides a more comprehensive coverage of the theoretical aspects of formal hardware verification.

1.1 Formal Verification in Hardware Design

Hardware design typically starts with a high-level specification, given in terms of block diagrams, tables, and informal text conveying the desired functionality. A combination of top-down and bottom-up design techniques are applied until a final design is obtained. *Validation* of the design involves checking that the physical design does indeed meet its specification. In a traditional design flow, this is accomplished through simulation and testing. Because exhaustive testing for nontrivial designs is generally infeasible, testing provides at best only a probabilistic assurance. *Formal verification*, in contrast to testing, uses rigorous mathematical reasoning to show that a design meets all or parts of its specification. This requires the existence of formal descriptions for both the specification and implementation. Such descriptions are given in notational frameworks with a formal semantics that unambiguously associates a mathematical object with each description, permitting these objects to be manipulated in a formal mathematical framework. Section 2 describes several notational frameworks, including predicate logic, temporal logic, and a variety of hardware description languages (HDLs).

The time required for formal verification must be considered when applying these techniques to a real project. There are highly automated formal methods comparable to traditional simulation in the time required and ease of use, but they tend to be limited to “small” designs. On the other end of the scale, formal verification frameworks exist that are in principle powerful enough to verify large, state-of-the-art designs [Brock et al. 1996]. Applying such a framework requires large amounts of time of highly skilled experts. Although formal verification methods are employed in the design of several state-of-the-art microprocessors and other complicated chips, we

are not aware of a complete top-to-bottom verification for such a design. The cost of such verification still appears to be prohibitive.

To achieve maximum leverage, formal methods must be applied in a way that ensures that they can keep up with the design flow [McMillan 1994]. If this is the case, formal methods can benefit the design process significantly, as they allow conceptual errors in the design to be uncovered early in the design process by verifying the high-level design against a set of requirements.

Formal methods can be beneficial even if a complete top-to-bottom verification is not carried out: the exercise of formalizing the requirements or a high-level specification can be useful in itself because it tends to clarify many aspects, such as boundary conditions, which are easily overlooked in informal specification [Miller and Srivas 1995]. Verifying properties of a high-level abstraction can catch many errors at an early stage of the design, avoiding costly corrections later. High-level descriptions can often be made concise enough to be tractable by automatic verification methods [McMillan 1994]. Finally, the cost of verification might be considered worthwhile for certain subsystems that are particularly difficult to design, while other “straightforward” modules can be treated with traditional methods.

1.2 Meaning of Verification

Formal methods are not a panacea. Even verifying that the netlist model of a design satisfies a formal specification does not provide complete assurance that the manufactured device will work as intended. The reasons for this are fundamental [Cohn 1989b]: there is no formalization of the intentions the designer had in mind; therefore one cannot formally verify that the top-level requirements specification faithfully captures this intention. Likewise, the formal interpretation of the lowest-level description is still an abstract model of the physical device. These models can be validated experimentally, but not verified formally. Furthermore, verification generally requires that some assumptions are made on the behavior of the environment in which a device is intended to operate. If the actual operating environment violates these assumptions, the device may fail despite successful verification. However, the application of formal methods can help make these assumptions explicit.

The literature contains many claims of the form “the XYZ device has been formally verified.” To be meaningful, such claims must be accompanied by a description of what properties were verified as well as a description of the form of the design description (e.g., RTL, abstract function blocks, behavioral models, gate, or switch level) that was used [Cohn 1989b]. For example, Brock and Hunt [1990] examined claims made about the application of formal methods to a commercial microprocessor noting many aspects of the design that were not verified.

1.3 Organization of the Paper

Section 2 introduces frameworks that are suitable for the formalization of specifications and implementation descriptions of hardware designs. Section 3 surveys methodologies and tools for the formal verification of assertions expressed in the introduced formalisms. Section 4 presents case studies of applications of formal verification to hardware design. We emphasize real, industrial designs in order to provide a realistic impression of verification problems that are tractable with current methods. We close the paper with a short summary and conclusions.

2. SPECIFICATION AND VERIFICATION CONDITIONS

There are two main approaches to the specification and corresponding verification of hardware. The first is concerned with specifying desired *properties* for the design. Formal verification is generally concerned with properties of a temporal nature, i.e., they do not pertain to static attributes of the system but rather characteristics of the system's behavior or execution. Temporal logics are a unifying framework for expressing such properties. Verification amounts to showing that all of the system's possible behaviors satisfy the temporal properties of its specification.

The second approach is based on specification in terms of a high-level model of the system. Here, the "good" behaviors of a system are given by the set of all behaviors of the high-level model, rather than a set of temporal properties. Verification then requires showing that each possible behavior of the system's implementation is consistent with some behavior of its high-level specification.

These two approaches are often used in conjunction: First, a high-level model of the design is shown to satisfy a set of desired temporal properties. Then, a series of more and more detailed specifications are developed, each of which is an implementation of the specification at the next higher level. In an appropriate technical framework, the temporal properties of the highest level model are preserved by the refinement steps and thus are satisfied by the lowest, most detailed, level. In this context, the first type of verification is also referred to as *design* or *property verification*, while the second form is known as *implementation verification*. Note that the distinction between the two forms is only a conceptual one. Both types of verification are instances of the same problem: the specification defines some constraint on the allowed behaviors of a system, and verification requires showing that the implementation meets this constraint.

In this section we introduce formalisms for both approaches. Section 2.1 is concerned with the specification of temporal properties, while Section 2.2 describes specification in terms of high-level models.

2.1 Specifications in Temporal Logic

In general, a temporal logic is a logic (usually propositional or first-order logic) augmented with temporal modal operators that allow reasoning about how the truth values of assertions change over time [Emerson 1990].

Temporal logic can express conditions such as “property p holds at all times” or “if p holds at some instant in time, q must eventually hold at some later time.” Propositions of this sort can be employed to specify desired properties of systems, such as “this bus controller will always grant at most one request for the bus at a time” and “every request will eventually be granted.”

The truth of a formula in temporal logic is understood relative to a system M perceived to be in one of a (possibly infinite) set of states S at any point in time, and which performs transitions between states as time progresses. *Atomic propositions* are associated with each state in S ; these form the basic building blocks for temporal formulas. In practice, the state space often consists of Boolean n -tuples; in this case a suitable form of the atomic propositions is “component i of the state tuple has value *true* (*false*).” The temporal operators permit the expression of relationships between the truth values of atomic propositions at different times; this corresponds to a statement about the truth values of atomic propositions as the system proceeds through a sequence of states. Pnueli [1977] first proposed using temporal logic to reason about system behaviors.

There are many different forms of temporal logics; in particular there are a number of options with regard to the underlying model of time, such as branching versus linear time, discrete versus continuous time, or the use of temporal operators on points versus intervals in time. In this section we first introduce Computation Tree Logic (CTL) as a representative example of temporal logics and demonstrate how it can be used for specification purposes. Later, we examine other variants of propositional temporal logics more briefly. For a more complete survey of temporal logics, see Pnueli [1986] and Emerson [1990].

There are three main considerations when choosing an appropriate logic: First, temporal logics differ in their expressiveness; there are properties that can be expressed in some logics, but not in other, less expressive, ones. Second, the complexity of the verification task depends on the choice of logic. Generally, it is more difficult or computationally expensive to show that a system satisfies a temporal formula in a more expressive logical framework. Finally, temporal logic specifications are often the top-level specification and need to be validated with respect to the informal intention the specifier has in mind. This is often easier in a simpler framework.

2.1.1.1 Computation Tree Logic. Computation Tree Logic (CTL) [Clarke et al. 1986] is a propositional logic of branching time; i.e., it is based on propositional logic and uses a discrete model of time where, at each instant, time may split into more than one possible future. We first introduce the syntax and formal semantics of CTL and then use a simple example to demonstrate how CTL can be used to specify desired behaviors of systems.

2.1.1.1.1 CTL Formulas and their Truth Semantics. The truth semantics of a CTL formula is defined with respect to a *temporal structure* $M = (S, R, L)$, where S is a set of states, $R \subseteq S \times S$ is a total binary relation

(i.e., $\forall_{s \in S} \exists_{t \in S} (s, t) \in R$), and $L : S \rightarrow 2^{\mathcal{P}}$ is a labeling of states with the atomic propositions in $\mathcal{P}$ that are true in a given state. R is the next-state relation of the structure, i.e., if the system is in state s at a given time instant, it will be in any of the states in the set $\{t \in S \mid (s, t) \in R\}$ at the following time instant. The totality requirement for R is included because CTL formulas have no sensible interpretation for states without successors. A *path* is defined as an infinite sequence of states $s_0, s_1, \dots$ such that $\forall_{i \geq 0} (s_i, s_{i+1}) \in R$.

Let $\mathcal{P}$ be a set of *atomic propositions*. CTL formulas are defined recursively:

- (1) Every atomic proposition $p \in \mathcal{P}$ is a CTL formula.
- (2) If f_1 and f_2 are CTL formulas, then so are $\neg f_1$, $f_1 \wedge f_2$, $\mathbf{AX}f_1$, $\mathbf{EX}f_1$, $\mathbf{A}[f_1 \mathbf{U} f_2]$, and $\mathbf{E}[f_1 \mathbf{U} f_2]$.

Intuitively, $\mathbf{AX}$ means “all successors”: the formula $\mathbf{AX}f_1$ holds in a system state s_0 iff f_1 holds in every successor state of s_0 . Likewise, $\mathbf{EX}$ means “there exists a successor.” The formula $\mathbf{A}[f_1 \mathbf{U} f_2]$ means “always-until,” i.e., along all possible paths f_1 holds until f_2 is satisfied. The formula $\mathbf{A}[f_1 \mathbf{U} f_2]$ holds in state s_0 iff, for every path starting from s_0 , $s = s_0, s_1, \dots$, there is some $i \geq 0$ such that f_2 holds in state s_i and f_1 holds in all states from s_0 to s_{i-1} . Note that $\mathbf{A}[f_1 \mathbf{U} f_2]$ holds in any state where f_2 holds. In the same fashion, $\mathbf{E}[f_1 \mathbf{U} f_2]$ means “exists-until,” i.e., there exists a path such that f_1 holds until f_2 is satisfied. Additional temporal operators are defined in terms of the ones above:

- (1) $\mathbf{AF}f = \mathbf{A}[\text{true} \mathbf{U} f]$ (f must hold *eventually*);
- (2) $\mathbf{EF}f = \mathbf{E}[\text{true} \mathbf{U} f]$ (there is a reachable state in which f holds);
- (3) $\mathbf{EG}f = \neg \mathbf{AF} \neg f$ (there is some path on which f always holds);
- (4) $\mathbf{AG}f = \neg \mathbf{EF} \neg f$ (f must always hold on all possible paths).

Consider a CTL formula f , a model structure $M = (S, R, L)$, and a state $s_0 \in S$. We denote the statement “ f holds in M at state s_0 ” with $M, s_0 \models f$. We write $s_0 \models f$ if the underlying model structure is understood; $M, S_0 \models f$ to abbreviate $\forall_{s \in S_0 \subseteq S} M, s \models f$; and $M \models f$ to abbreviate $M, S \models f$. The relation $\models$ defines the formal truth semantics for CTL and is defined recursively as follows:

- (1) $M, s_0 \models p$ iff $p \in L(s_0)$.
- (2) $M, s_0 \models \neg f$ iff not $(M, s_0 \models f)$.
- (3) $M, s_0 \models f_1 \wedge f_2$ iff $(M, s_0 \models f_1)$ and $(M, s_0 \models f_2)$
- (4) $M, s_0 \models \mathbf{AX}f$ iff $\forall_{t \in S} (s_0, t) \in R \Rightarrow (M, t \models f)$.

- (5) $M, s_0 \models \mathbf{EX}f$ iff $\exists_{t \in S}(s_0, t) \in R \wedge (M, t \models f)$.
- (6) $M, s_0 \models \mathbf{A}[f_1 \mathbf{U} f_2]$ iff for all paths $s_0, s_1, \dots$,
 $\exists_{i \geq 0}(M, s_i \models f_2) \wedge \forall_{j=0 \dots i-1} M, s_j \models f_1$.
- (7) $M, s_0 \models \mathbf{E}[f_1 \mathbf{U} f_2]$ iff for some path $s_0, s_1, \dots$,
 $\exists_{i \geq 0}(M, s_i \models f_2) \wedge \forall_{j=0 \dots i-1} M, s_j \models f_1$.

The semantics of CTL were also defined with respect to model structures that correspond to sequential circuits, modeled as deterministic Moore machines [Coudert and Madre 1990]. In this case, the behavior of the system is governed by a transition *function* and nondeterminism arises only from the circuit's inputs. The existential and universal quantifiers range over input symbols (in a completely deterministic system they would be equivalent). Note that the above semantics for CTL, which applies to closed systems only, can be extended to open systems with inputs by augmenting the state space with a component representing the current input, and extending the transition relation to allow nondeterministic changes to this component [Grumberg and Long 1994].

2.1.1.2 Specifications in CTL. We now introduce a simple system that is used as an example throughout this paper. Consider the controller of a (very abstract) traffic-light at a four-way intersection where the lights in opposing directions always show the same color. Let $\mathcal{C} = \{r, y, g\}$ be the set of traffic-light colors. The state space of the controller can be modeled as $S = \mathcal{C} \times \mathcal{C}$, where the first (second) component denotes the color of the lights in the north-south (east-west) direction. Figure 1 shows the state-transition graph of the traffic-light controller. The corresponding state-transition relation R is defined such that $(s, t) \in R$ iff there exists an arc from state s to state t in the graph. The states on the right are states we expect a correct and properly initialized implementation of a traffic-light not to reach. We added self-loops to each of them to ensure that R is indeed a total relation. A suitable set of atomic propositions is given by

$$L((c, d)) = \{[ns = c], [ew = d]\}.$$

For example, the atomic proposition $[ns = g]$ holds in states (g, r) , (g, y) , and (g, g) . Let $M = (S, R, L)$ be the temporal structure corresponding to our traffic-light controller.

There are two important classes of properties that one usually wants to verify for a given system [Owicki and Lamport 1982]:

Safety properties. These are properties that intuitively assert that “bad things never happen.”

Liveness properties. Properties in this class state that “good things happen eventually.” They are also referred to as *eventuality* or *progress* properties.

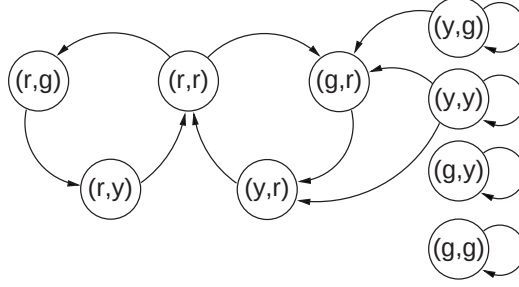


Fig. 1. State transition graph for the traffic-light controller.

For a more rigorous classification of liveness and safety properties, see Pnueli [1986] and Emerson [1990].

An important safety property of our traffic-light is that it shows a red light in at least one direction at all times. In CTL, this property can be expressed as $P = \mathbf{AG}([ns = r] \vee [ew = r])$. Note that $M \not\models P$ since, for example, $M, (g, y) \not\models P$. However, one can easily be convinced by exploring all possible state transitions that, e.g., $M, (r, r) \models P$. An equivalent formulation is

$$M \models ([ns = r] \wedge [ew = r]) \Rightarrow \mathbf{AG}([ns = r] \vee [ew = r]).$$

This specification is in a commonly encountered generic form of safety properties, $M \models P_{init} \Rightarrow \mathbf{AG}P_s$, where P_{init} and P_s are *instantaneous formulas* not containing temporal operators. A safety property of this form has the intuitive interpretation that every computation of M , which starts in a state satisfying P_{init} also satisfies P_s at all times.

A desirable liveness property of the traffic-light is that a particular direction will “always eventually” see green lights. This property can be expressed in CTL as

$$Q = \mathbf{AG}(\mathbf{AF}[ns = g]) \wedge \mathbf{AG}(\mathbf{AF}[ew = g]).$$

However, $M, (r, r) \models Q$ does not hold, because the transition relation R specifies two successor states for (r, r) . This means that R allows, for example, the path that repeatedly cycles through the three states (r, r) , (r, y) , (r, g) , $\dots$. Such a path is a counterexample for the first conjunct of Q . We can phrase a weaker property

$$Q' = \mathbf{AG}(\mathbf{EF}[ns = g]) \wedge \mathbf{AG}(\mathbf{EF}[ew = g]),$$

such that indeed $M, (r, r) \models Q'$. Q' states that, from all states on all paths, M *permits* a sequence of transitions to a state in which a particular direction sees green.

The stronger liveness property Q holds for M only under additional fairness assumptions stating that M does not consistently “discriminate”

against state transitions. However, it can be shown that liveness under fairness assumptions cannot be expressed in CTL¹(see Emerson and Halpern [1986] and Emerson [1990]). In the following section, we consider more general branching time logics that permit such assertions.

2.1.2 More Expressive Branching Time Logics. Emerson and Halpern [1986] define the logic CTL* as a generalization of CTL where a path quantifier may be followed by arbitrary temporal formulas. Formally, CTL* formulas are defined recursively in terms of *path formulas* and *state formulas*, as follows:

- (1) Any atomic proposition is a state formula.
- (2) If s_1, s_2 are state formulas and p is a path formula, then $s_1 \wedge s_2, \neg s_1$ and $\mathbf{E}p$ are state formulas.
- (3) If s is a state formula and p_1, p_2 are path formulas, then $s, p_1\mathbf{U}p_2, \neg p_1, p_1 \wedge p_2, \mathbf{X}p_1$ are path formulas.
- (4) Any state formula is a CTL* formula.

In addition, the abbreviations $\mathbf{F}p = \text{true}\mathbf{U}p$, $\mathbf{G}p = \neg \mathbf{F} \neg p$, and $\mathbf{A}p = \neg \mathbf{E} \neg p$ are introduced. Informally, the truth of a state formula is defined in the obvious way for the Boolean operators, and such that $\mathbf{E}p$ holds in a state r if there is a path starting with r that satisfies p . The truth of a path formula is defined with respect to a given path in a natural way, for example, $p_1\mathbf{U}p_2$ holds for a path q if p_1 holds on a finite and possibly empty prefix of q , followed by a state in which p_2 holds. A path formula that is a state formula s holds for a path q iff s holds in the first state of q . For a formal treatment of the semantics of CTL*, see Emerson and Halpern [1986] and Emerson [1990].

The path quantifiers $\mathbf{E}$ and $\mathbf{A}$ apply to a path formula and result in a state formula. These quantifiers transform a statement about computations starting from a state into an assertion about the state itself. In particular, $\mathbf{E}p$ holds for state s_0 if there is a path $s_0, s_1, s_2, \dots$ such that p holds for this path. Furthermore, if we constrain the syntax of CTL* such that path quantifiers must be followed immediately by one of the modalities $\mathbf{X}, \mathbf{U}, \mathbf{F}$ or $\mathbf{G}$, we obtain the logic CTL embedded in CTL*. The essential difference between CTL* and CTL is that all subformulas of a CTL formula must be state formulas; this precludes assertions about individual computations.

Liveness properties under fairness assumptions can, for example, be expressed in CTL* in the form

$$\mathbf{A}((\mathbf{G}\mathbf{F}p_1 \wedge \dots \wedge \mathbf{G}\mathbf{F}p_n) \Rightarrow \mathbf{F}q). \quad (1)$$

¹Note however that the semantics of CTL can be adapted to accommodate fairness (see the following section).

Here, q is a property that must eventually hold on every path that “always eventually” satisfies each of the properties $p_1, \dots, p_n$. The key point is that the subformulas $\mathbf{GF}p_i$ and $\mathbf{F}q$ all refer to the *same* path, which can be thought of as being “bound” by the path quantifier $\mathbf{A}$. As mentioned above, CTL cannot express assertions about paths; this gives an intuitive explanation why liveness properties such as (1) cannot be stated in CTL.

In the traffic-light example, we could require the fairness assumption that the controller “always eventually” takes each of the two transitions from (r, r) . The formulas $[ns = r] \wedge [ew = r] \wedge \mathbf{X}[ew = g]$ and $[ns = r] \wedge [ew = r] \wedge \mathbf{X}[ns = g]$ hold for a path starting in (r, r) if this path’s first transition is $(r, r) \rightarrow (r, g)$ or $(r, r) \rightarrow (g, r)$, respectively. Thus, the fairness assumption can be stated as

$$T = \mathbf{GF}([ns = r] \wedge [ew = r] \wedge \mathbf{X}[ew = g]) \wedge \\ \mathbf{GF}([ns = r] \wedge [ew = r] \wedge \mathbf{X}[ns = g]).$$

The desired liveness property does indeed hold under this fairness assumption:

$$M, (r, r) \models \mathbf{A}(T \Rightarrow (\mathbf{F}[ns = g] \wedge \mathbf{F}[ew = g])).$$

It is not always necessary to resort to full CTL^{*} to express certain notions of fairness. Clarke et al. [1986] propose the logic CTL^F that has the same syntax as CTL. However, *fairness constraints* for a model can be stated in terms of sets of states that have to be visited infinitely often on paths that are to be considered fair. The semantics of CTL^F is defined in exactly the same way as CTL, except that all path quantifiers range over fair paths only.

The logics $\forall$ CTL and $\forall$ CTL^{*} are defined as subsets of CTL and CTL^{*}, respectively, free of existential path quantifiers [Lamport 1980].

2.1.3 Linear Time Logic. In the branching time framework described in the previous sections, the nondeterminism of a system is captured by allowing for more than one future at any time instant. An alternative view is to regard time as linear and to consider only the one future or execution that “actually takes place” [Lamport 1980]. Thus, a linear time temporal formula is an assertion about one particular sequence of states; to reason about nondeterministic systems it is necessary to consider more than one execution.

Syntactically, formulas of linear-time logic (LTL) look like path-quantifier-free CTL^{*} formulas. The semantics of an LTL formula is defined with respect to a *linear-time structure* $M = (S, x, L)$, where S is a set of states, $x = x_0, x_1, \dots$ is an infinite sequence of states, and $L : S \rightarrow 2^{\mathcal{P}}$ is a labeling of states with atomic propositions in $\mathcal{P}$ [Emerson 1990]. Essentially, the truth of an LTL formula with respect to a sequence is defined in

the same manner as truth of a CTL* path-formula is defined with respect to a path.

Lamport [1980] uses linear-time structures of the form $M_X = (S, X, L)$, where S and L are defined as above and X is a set of sequences of states corresponding to the set of all possible computations of a system. In this framework, a LTL formula p holds for a structure M_X , denoted $M_X \models p$, iff p holds for all $x \in X$. We write $M_X, x \models p$ to denote that p holds for a particular $x \in X$.

The truth of an LTL formula can also be extended to temporal structures $M_R = (S, R, L)$, where R is a transition relation (see Section 2.1.1). Let $M_{R,X} = (S, X_R, L)$, where $X_R = \{x \mid x \text{ is a path of } M_R\}$, denote the corresponding linear-time structure. Then, $M_R \models \varphi$ iff $M_X \models \varphi$, i.e. $x \models \varphi$ for all paths x generated by M_R . The universal quantification implicit in the notation $M_{R,X} \models p$ can be made explicit by considering the CTL* formula $\mathbf{Ap}$ with respect to M_R , interpreted as a branching-time structure [Emerson and Halpern 1986].

The main difference between linear- and branching-time logic (BTL) is the lack of existential quantification over paths in LTL. Emerson and Halpern [1986] have shown that CTL* is strictly more expressive than LTL, i.e., there are properties such as $\mathbf{EF}p$ that cannot be expressed in LTL. There is some argument as to whether existential path quantification is essential in specifications or should not be used at all. Emerson and Halpern [1986] argue that existential quantification plays an important role in reasoning about concurrent systems. Others, however, point out that specifications should capture properties of all possible computations of a system, not only some, in the sense that one usually wants to specify what a system *must* do and not what it only *might* do [Clarke et al. 1994].

Note that LTL and BTL are equivalent if the model structure is deterministic, in which case it gives rise to exactly one computation.²

2.1.4 μ -Calculus. An alternative characterization of temporal modalities can be given in terms of fixpoints of monotonic predicate transformers. Let $M = (S, R, L)$ be a temporal structure as above. Consider the set of predicates on states and let each predicate be represented by the subset of $P \subseteq S$ such that the predicate is true in, and only in, the states in P . Let $\mathbf{AXP}$ denote the set of states such that all of those state's successor states are in P , i.e.,

$$\mathbf{AXP} = \{s \in S \mid \forall_{s' \in S} (s, s') \in R \Rightarrow s' \in P\}. \quad (2)$$

Consider the sequence of sets defined by

$$Q_0 = S,$$

²This assumes that the model structure is closed; thus no nondeterminism is introduced through input signals (see Section 2.1.1.1).

$$Q_{i+1} = P \cap \mathbf{AX}Q_i.$$

The set Q_i , $i > 0$ has the property that for every state $s \in Q_i$, P holds, and for all successors s' of s , P holds on a prefix of length $i - 1$ of all paths starting in s' . In other words, P holds on any path prefix of length i starting from a state $s \in Q_i$. One can show that the greatest fixpoint $\nu Q.P \cap \mathbf{AX}Q$ exists and that

$$Q_\infty = \lim_{i \rightarrow \infty} Q_i = \nu Q.P \cap \mathbf{AX}Q, \quad (3)$$

(see Emerson [1990]). We can interpret Q_∞ as the set of states such that for any state $s \in Q_\infty$, P holds on all states of all paths starting in s . This justifies the definition $\mathbf{AG}P = \nu Q.P \cap \mathbf{AX}Q$, which provides a characterization of the temporal operator $\mathbf{AG}$ in terms of a fixpoint. Similar fixpoint characterizations can be derived for the other temporal operators.

Propositional μ -calculus [Kozen 1993; Bradfield 1992] is a temporal logic based on fixpoint characterizations. Formulas of the μ -calculus are composed of simple predicates, the next-state operators on sets $\mathbf{EX}$ and $\mathbf{AX}$, Boolean connectives, and the least and greatest fixpoint operators μ and ν . The semantics of a formula with respect to a structure M is given in terms of sets of states in a similar fashion as above. Thus, the denotation of a formula f is a subset $Q_f \subseteq S$, which is intuitively the set of states in which f holds. The possibility of nesting the fixpoint operators provides considerable expressive power. In particular, μ -calculus subsumes CTL^* and LTL [Emerson 1990].

The fixpoint characterizations for the temporal operators are of practical importance, even if one does not require the full expressive power of the μ -calculus. For a structure M whose set of states S is finite, the sequence Q_i , $i = 0, 1, \dots$ in Eq. (3) must converge in a finite number of steps. Thus, the iterative fixpoint computation provides the basis for an algorithm that computes the set of states satisfying a temporal formula in, e.g., CTL or any other temporal logic that can be expressed in the μ -calculus. This approach is explored in greater detail in Section 3.1.2

2.1.5 Invariants and Safety Properties. In the previous sections, we introduced some variants of temporal logic with rather comprehensive expressiveness. In many applications, however, it is sufficient, and often more intuitive, to specify a system in terms of rather simple assertions.

One of the simplest forms of temporal specification is in terms of invariants and safety properties. An invariant I of a structure M is a property such that for all states of M that satisfy I , their successor states also satisfy I . This can be expressed in CTL as $M \models (I \Rightarrow \mathbf{AX}I)$. One can easily verify that, e.g., $I_1 = [ns = r] \vee [ew = r]$ and $I_2 = [ns = g] \wedge \neg [ew = r]$ are invariants of the traffic-light controller. In Section 2.1.1.2,

we already encountered the traffic-light's safety property $P_{init} \Rightarrow \mathbf{AG}P_s$, with $P_{init} = [ns = r] \wedge [ew = r]$ and $P_s = [ns = r] \vee [ew = r]$.

There is an important connection between invariants and safety properties: Given

- (1) $M \models P_{init} \Rightarrow I$,
- (2) $M \models I \Rightarrow \mathbf{AX}I$,
- (3) $M \models I \Rightarrow P_s$,

we can conclude $M \models P_{init} \Rightarrow \mathbf{AG}P_s$ using a simple induction argument over reachable states. This allows safety properties to be established inductively, where the inductive step corresponds to showing that I is an invariant [Lamport and Schneider 1984]. In the traffic-light example, once I_1 has been shown to be an invariant, simple Boolean algebra suffices to show $P_{init} \Rightarrow I_1$ and $I_1 \Rightarrow P_s$, which together establishes the desired safety property.

Staunstrup and Møllergaard [1995] present a framework where properties of modular designs are expressed in terms of invariants of individual components as well as the entire system. Verification proceeds by proving invariants of individual modules and combining the results to prove invariants of their composition.

2.1.6 Trajectory Formulas. Trajectory formulas (TF) form a restricted temporal logic that offers only the next-time operator and does not allow negation or disjunction [Seger and Bryant 1995]. Trajectory formulas are defined recursively as follows :

Simple predicates. For simplicity, we consider the case where TF are used to assert properties of digital circuits. If nd is a node in the circuit model, then nd **is** 0 and nd **is** 1 are trajectory formulas. More general models are possible, but require appropriately defined simple predicates.

Conjunction. If F_1, F_2 are trajectory formulas, then so is $(F_1 \wedge F_2)$.

Next time. If F is a trajectory formula, then so is $\mathbf{NF}$.

Domain restriction. If F is a trajectory formula and E is a Boolean expression over symbolic variables, then F **when** E is a trajectory formula.

A *trajectory assertion* has the form $[A \Rightarrow C]$, where A and C are trajectory formulas. Informally, a trajectory assertion $[A \Rightarrow C]$ holds for a model structure M , written $M \models [A \Rightarrow C]$ iff each sequence of states of M that satisfies the *antecedent* A also satisfies the *consequent* C . Trajectory assertions can be efficiently verified using techniques described in Section 3. Typically, A specifies constraints on how the inputs of a circuit are driven, while C asserts the expected results on the output nodes. For

example, the formula $[in \text{ is } 0 \Rightarrow N(out \text{ is } 1)]$ asserts that from any state in which the node *in* is 0, the node *out* must be 1 in the next state.

Rather than writing a separate assertion for every possible combination of inputs, symbolic variables may be used to write much more succinct specifications. For example, an inverter with a delay of one time unit can be specified using the symbolic variable *a* and the symbolic trajectory assertion $[in \text{ is } a \Rightarrow N(out \text{ is } \neg a)]$. Here, we used the abbreviation

$$nd \text{ is } b = ((nd \text{ is } 0) \text{when } \neg b) \wedge ((nd \text{ is } 1) \text{when } b).$$

A trajectory assertion is implicitly universally quantified over any symbolic variables that appear in it.

As a somewhat larger example, consider a correlator with inputs *x* and *y* and output *out*. The output should be true if and only if the previous three inputs on *x* matched the corresponding inputs on *y*. This behavior is specified by the assertions (we write N^k , where *k* is a constant, to denote *k* applications of the next time operator):

$$[(x \text{ is } a) \wedge (y \text{ is } a) \wedge (N(x \text{ is } b) \wedge N(y \text{ is } b)) \wedge (N^2(x \text{ is } c) \wedge N^2(y \text{ is } c)) \\ \Rightarrow N^3(out \text{ is } 1)] \text{ and}$$

$$[(x \text{ is } a) \wedge (y \text{ is } \neg a) \Rightarrow N(out \text{ is } 0) \wedge N^2(out \text{ is } 0) \wedge N^3(out \text{ is } 0)]. \quad (4)$$

The first assertion states that if *x* and *y* are the same for three consecutive cycles, then the output must be 1 on the next cycle. The second assertion states that if *x* and *y* differ on some cycle, then the output must be 0 on the next three cycles. No initial state is specified with a trajectory assertion, therefore verifying such an assertion shows that it holds everywhere along a trajectory. For example, the specification for the correlator describes its operation for all times (although the value of *out* is not fully specified for the first three time steps).

2.2 Specification with High-Level Models

The previous section presented a variety of approaches for specifying a system in terms of *desired properties*. An alternative style of specification uses a high-level model to stipulate the allowed behaviors of a system. In this framework, verification entails reasoning about the relationship between the high-level model M_S , also referred to as the *specification*, and a lower-level model M_I , the *implementation*.

Central to this approach is the notion of *abstraction*, which permits unnecessary detail to be hidden from the high-level model. Furthermore, specifications may be given in an hierarchical fashion; starting from a very abstract model at the highest level, one proceeds through a series of abstractions to a detailed description of the implementation. The design at

some level k assumes the role of the implementation with respect to the specification at level $k - 1$, and the role of the specification for level $k + 1$.

To reason about the implementation and the specification, formal interpretations must be given for their descriptions. There are three common approaches to this formalization: A device's behavior can be specified in terms of functions and predicates of standard first or higher-order predicate logic. Alternatively, its behaviors can be described by a state transition system, whose definition in turn can be expressed in languages such as CSP or UNITY. Finally, the description may be in terms of languages recognized by automata on infinite objects.

In this section, we first review the role of abstraction and then describe how specifications are described both in logic and as transition systems. We then survey a number of formalizations of what it means that " M_I implements M_S ," and review automata-theoretic approaches.

2.2.1 Abstraction Mechanisms. Melham [1988] identifies four different types of abstraction prevalent in hardware verification. We illustrate each category using a simple, 3-bit, up-down counter as an example.

Structural abstraction suppresses details about the implementation's internal structure in the specification. The specification gives a "black-box" view of the design that reflects the system's externally observable behavior without constraining its internal design. For example, the counter can be described as a component with inputs for the clock and the direction control, and an output for the value of the count. A structural refinement of this description could describe the counter using flip-flops and gates.

Behavioural abstraction suppresses details about what the component does under operating conditions that should never occur. For example, a behavioral description of the three-bit counter may omit a specification of what the counter does if incremented from its maximum count or decremented from its minimum count. Behavioral abstraction may also indicate "don't care" conditions. For example, if several devices are requesting access to a bus and they all have the same priority, the bus controller may be allowed to grant the requests in an arbitrary order. This gives the designer greater flexibility to optimize implementation. The specification is more abstract, in the sense that its behaviors are a superset of the implementation behaviors.

Data abstraction relates signals in the implementation to signals in the specification when they have different representations. For example, the specification of the counter may specify that the output is an integer between zero and seven, and the implementation may have an output that consists of three Boolean-valued signals. Data abstraction requires a mapping that determines how the states or signals of the implementation are to be interpreted in the specification's semantic domain.

Temporal abstraction relates time steps of the implementation to time steps of the specification. For example, the counter may be specified in terms of what happens for each clock cycle while the implementation uses a

two-phase clock. Frequently, specifications of a microprocessor use the execution of a single instruction as the basic unit of time. An implementation, however, would base its notion of time on the execution of a microcode instruction, a clock cycle, or a clock phase. Temporal abstraction requires that corresponding execution states at the two time scales be identified. This is complicated by the possibility that one specification time-unit does not always necessarily correspond to the same number of implementation steps.

2.2.2 Specifications in Logic. Gordon [1985] has argued that it is not strictly necessary to resort to specialized hardware description languages; rather, formal logic is sufficient to specify hardware systems. In the following, we assume that the reader is familiar with first- and second-order predicate logic (see, e.g., Kleene [1967]). We use the usual notation for predicate logic, plus the following notation for conditional expressions,

$$(a \rightarrow b | c) = \begin{cases} b & \text{if } a, \\ c & \text{otherwise,} \end{cases}$$

where a is of Boolean type and b and c share the same type.

The external behavior of a device can be modeled as a predicate over the device's external connections or its externally visible state. If the device's timing behavior is irrelevant, a first-order predicate suffices and the external connections are modeled as values in an appropriate domain, such as Booleans or integers. For example, the definition

$$NAND(i_1, i_2, o) = (o = \neg (i_1 \wedge i_2))$$

specifies a delay-less NAND gate by constraining its external connection o (the output) to be equal to the NAND of its (input) wires i_1, i_2 .

If time has to be taken into account and a discrete model of time suffices, it is convenient to represent signals as sequences of values, i.e., as functions from natural numbers into the underlying domain. A NAND gate with a switching delay of δ time units can be specified with the following predicate:

$$NAND(i_1, i_2, o) = \forall_{t \in \mathbb{N}} o(t + \delta) = \neg (i_1(t) \wedge i_2(t)).$$

Here, $NAND$ is a second-order predicate on functions from natural numbers to Booleans (i.e. the sequences i_1, i_2 , and o). The composition of devices thus specified into more complex devices is formalized by identifying the internal connections and hiding them by existential quantification. For example, we can describe an AND gate built from two NAND gates, as shown in Figure 2 with the predicate

$$AND(i_1, i_2, o) = \exists_q NAND(i_1, i_2, q) \wedge NAND(q, q, o).$$

A synchronous device can be specified in terms of a predicate over the sequences of its own states and its environment's states. Let $\mathcal{S}_D$ denote the device's state space, and $\mathcal{S}_E$ the state space of its environment. A sequence of states, or *state stream* [Windley 1995a], is modeled as a function from natural numbers into states.

As an example, we consider the specification of a modulo-eight counter. The counter's state space is given by

$$\mathcal{S}_D = \{i \in \mathbb{N} \mid 0 \leq i < 8\}.$$

The counter's environment $\mathcal{S}_E = \{\text{true}, \text{false}\}$ consists of a reset signal; a value of *true* on this signal causes the counter to reset to zero. Otherwise, the counter increments modulo 8 at each time instant.

Since the counter is a deterministic device, we can give its next-state function $N : (\mathcal{S}_D, \mathcal{S}_E) \rightarrow \mathcal{S}_D$ as follows:

$$N(c, \text{res}) = \text{res} \rightarrow 0 \mid (c = 7 \rightarrow 0 \mid c + 1).$$

We can now define a predicate which constrains a state stream $s : \mathbb{N} \rightarrow \mathcal{S}_D$ and an environment stream $e : \mathbb{N} \rightarrow \mathcal{S}_E$ to behaviors that are consistent with the counter's next state function:

$$\text{Counter}(s, e) = \forall_{t \in \mathbb{N}} s(t + 1) = N(s(t), e(t))$$

The use of second-order logic in the above specification is not essential; the relevant aspects of the specification are captured in the definition of the next-state function N which does not depend on higher-order logic [Hunt 1989].

The use of a general-purpose logic as a specification language has the advantage that one can immediately associate a formal interpretation with the specification based on the formal semantics of the logic. This has to be contrasted with the use of specialized hardware specification languages, which are often not defined with formalization in mind; for instance, the definition of a formal semantics for VHDL is an area of active research [Van Tassel 1992; Déharbe and Borriane 1995; Kloos and Breuer 1995].

However, to produce a lucid specification in logic may require greater discipline on the part of the specifier. This can be aided by specification frameworks that constrain the specification of the next-state function to have a certain structure (see Section 4.1.1).

2.2.3 Specification Using Transition Systems. It is clear that we can specify the allowed behaviors of a device in terms of a state transition system whose transitions correspond to the behaviors we have in mind. For example, the state transition system for the traffic-light controller in Section 2.1.1.2 can be viewed as an abstract specification for a “real” controller, which could for instance be implemented as a synchronous design.

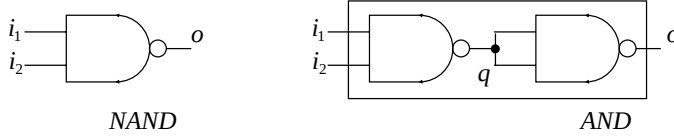


Fig. 2. (a) NAND gate; (b) AND gate.

Formally, a state transition system is a tuple $M = (S, S_0, R)$, where S is a set of states, $S_0 \subseteq S$ the set of initial states, and $R \subseteq S \times S$ is the transition relation. We described the traffic-light specification in terms of a state transition graph, which immediately gives rise to a transition system as defined above, once the initial states are identified. For larger applications, graph representations become difficult to handle and a textual representation is often more appropriate. Nevertheless, graphical representations such as Statecharts, which facilitate a hierarchical design description [Harel 1988], can contribute to a better understanding of a system and its properties.

A variety of languages suitable for specifying transition systems have been proposed in the literature. Here, we present UNITY as a representative example [Chandy and Misra 1988]. The basic building block of a UNITY program is the guarded assignment. For example, the assignment

$$ns := grn \text{ if } (ns = red) \wedge (ew = red)$$

is enabled if the variables ns and ew are both equal to *red*, and if so, will set ns to *grn*. The assignment may be executed only if it is enabled, and if so, is executed as a single atomic action. Although other statements may be concurrently enabled, executions can only reach states that are reachable by performing one statement at a time. UNITY is different from sequential PASCAL-like languages, in that it neither provides sequential composition of assignments nor control structures such as loops or branches.

Figure 3 shows a UNITY program for the traffic-light controller. The **declare** section declares two variables, ns and ew , of an enumerated type with the three different values *red*, *grn*, and *yel*. The two variables represent the states of the lights in the two directions. The **initially** section specifies that the initial values of both variables are *red*. Variables not explicitly mentioned in this section have arbitrary initial values. Finally, the **assign** section of the program declares a list of assignments that are combined with the asynchronous combinator $[]$. The execution semantics of a UNITY program consisting of $[]$ -composed assignments is such that, for a given state, any one of the assignment statements enabled in that state is selected nondeterministically and executed, which results in a state change. This process is repeated indefinitely. For example, in state $(ns = red, ew = red)$, a nondeterministic choice between the first and the fourth assignment statement takes place. This corresponds to the state having two successor states in the graph representation in Figure 1.

```

program TrafficLight
  declare
    ns, ew : {red, grn, yel}
  initially
    ns, ew = red, red
  assign
    ns := grn   if (ns = red) ∧ (ew = red)
    [] ns := yel   if (ns = grn)
    [] ns := red   if (ns = yel)
    [] ew := grn  if (ns = red) ∧ (ew = red)
    [] ew := yel  if (ew = grn)
    [] ew := red  if (ew = yel)
end TrafficLight
    
```

Fig. 3. UNITY program for the traffic-light controller.

A UNITY program P denotes a state transition system $M = (S, S_0, R)$ whose state space S is given by the cross-product of the domains of the variables declared in P , whose initial states S_0 are as specified in the **initially** section, and whose transition relation is given by the guarded multiassignments of P . A UNITY program can also be seen as denoting a temporal structure by augmenting the corresponding transition system with atomic propositions of the form $[v = a]$, where v is a variable of P and a is a value in v 's domain. This facilitates a textual notation for systems whose specification is given in terms of temporal formulas as well.

Other languages that provide suitable notation for high-level design descriptions and for temporal structures include CSP [Hoare 1978]; Mur ϕ [Dill et al. 1992; 1996]; SMV [McMillan 1992]; SPL [Manna and Pnueli 1992; Manna et al. 1994]; and Synchronized Transitions [Staunstrup 1994]. Traditional HDLs such as Verilog or VHDL can be used as well; this often requires the restriction of the language to a subset to make the definition of a formal semantics tractable.

In formal verification, nondeterminism is frequently used for behavioral abstraction, leaving aspects of a design unspecified at a certain level of abstraction. For example, the traffic-light controller depicted in Figure 1 does not specify whether the (r, g) or (g, r) state will follow the (r, r) state. The specification only requires that when a light cycles, it must go from red to green to yellow and back to red. In particular, this specification is satisfied both by an implementation that strictly alternates between the two roads and by one that can show green to the same road more than once based on input from a sensor loop.

2.2.4 Refinement. The notion that M_I implements M_S is formalized in terms of a *refinement* relation $\sqsubseteq$. In this section we describe several definitions of this relation, which have been used in the literature. It is generally accepted that $M_I \sqsubseteq M_S$ should imply that each observable behavior of M_I is also an observable behavior of M_S [Gerth 1989]. Generally, the equivalence of observable behaviors is understood with respect to an

appropriate abstraction mapping that relates corresponding aspects of M_I and M_S .

Abadi and Lamport [1991] describe a very general notion of refinement based on trace inclusion. Each state has two components: an “external” (i.e., visible) part and an “internal” (i.e., hidden) part. Let E denote the set of externally visible states of both the implementation and the specification; data abstraction between the two levels is handled by mapping external implementation states into their corresponding specification states. Let I_I and I_S denote the internal states of M_I and M_S , and let $S_I = E \times I_I$ and $S_S = E \times I_S$ be the state spaces of the two models.

Let X_I and X_S denote the sets of all sequences of states allowed by M_I and M_S . Then, M_I is defined to be a refinement of M_S iff

$$\begin{aligned} \forall_{\sigma \in X_I} (\sigma = (e_0, q_0), (e_1, q_1), (e_2, q_2), \dots \\ \Rightarrow \exists_{\sigma' \in X_S} \sigma' = (e_0, r_0), (e_1, r_1), (e_2, r_2), \dots) \end{aligned}$$

Thus, $M_I \sqsubseteq M_S$ iff for each behavior of the implementation, there exists a specification behavior with the same externally visible states. Temporal abstraction is accommodated by allowing *stuttering*; it is assumed that for every $\sigma \in X_S$, all sequences obtained by replacing states with finitely many repetitions thereof, or by removing repetitions, are also in X_S . For example, consider the implementation behavior

$$\sigma_1 = (e_0, q_0), \dots, (e_0, q_{k-1}), (e_1, q_k), \dots$$

whose first k states differ only in their internal part. Assume that the first k transitions implement a single transition at the specification level, from state (e_0, r_0) to (e_1, r_1) . Then σ_1 can be matched up with the “stuttered” specification behavior

$$\sigma_2 = (e_0, r_0), \dots, (e_0, r_0), (e_1, r_1), \dots$$

Maretti [1994] introduces a generalization of trace inclusion where the externally visible state of the system is subject to an interface protocol. Consider, for example, a device that outputs data on a bus and indicates the availability of valid data on the bus by setting a signal *valid* to *true*. An interface protocol can be defined that specifies that the bus is only to be observed (i.e., is externally visible) if *valid* = *true*. An implementation of the device that places arbitrary values on the bus while *valid* = *false* can still be regarded a refinement of the specification with respect to the interface protocol, even though the values on the bus are not consistent with the specification if the interface protocol is disregarded.

In a framework where the specification and the implementation are described in predicate logic, refinement is usually expressed in a form that is equivalent to the following [Melham 1988; Windley 1995b]:

$$\forall_{s:N \rightarrow S_I} \forall_{e:N \rightarrow S_E} Imp(s, e) \Rightarrow Spec(A_s(s), A_e(e)). \quad (6)$$

This assertion states that for each pair (s, e) of implementation-level state and environment streams consistent with the implementation, the streams $A_s(s)$ and $A_e(e)$ obtained by subjecting s and e to their respective abstraction functions are consistent with the specification. This notion of refinement implies trace inclusion, it is however more restrictive because it requires the abstraction mapping to be a function and does not allow more general relations. In addition to modeling data abstraction, this approach can model temporal abstraction by mapping a sequence of implementation states to the same specification state. Thus, the finer representation of time in the implementation appears as stuttering moves for the specification. With this sense of abstraction, every safety property of the specification has a corresponding property for the implementation. The same cannot be said of liveness properties, as the abstraction mapping does not ensure fairness. For a more detailed exposition on abstraction, the reader is referred to Abadi and Lamport [1991] and Windley [1995a].

2.2.5 Automata on Infinite Objects and Language Containment. An ω -automaton (see, e.g., Thomas [1990]) is a tuple $\mathcal{A} = (\Sigma, Q, \bar{q}, \Delta, F)$, where Σ is a finite alphabet, Q a finite set of states, $\bar{q} \in Q$ the initial state, $\Delta \subseteq Q \times \Sigma \times Q$ a transition relation, and F a fairness constraint. Let $\Sigma^\omega = \{\alpha_0\alpha_1\alpha_2 \dots \mid \alpha_i \in \Sigma, i \geq 0\}$ denote the set of infinite sequences (or ω -words) over Σ . A run of $\mathcal{A}$ on an ω -word $\alpha_0\alpha_1\alpha_2 \dots \in \Sigma^\omega$ is a sequence of states $q = q_0q_1q_2 \dots$ with $q_i \in Q$ such that $q_0 = \bar{q}$ and $(q_i, \alpha_i, q_{i+1}) \in \Delta$ for $i \geq 0$. A run q is called *successful* if it satisfies the fairness constraint. $\mathcal{A}$ is said to *accept* a word $\alpha \in \Sigma^\omega$ iff there exists a successful run of $\mathcal{A}$ on α . Let $\mathcal{L}(\mathcal{A}) = \{\alpha \in \Sigma^\omega \mid \mathcal{A} \text{ accepts } \alpha\}$ denote the *language* of $\mathcal{A}$.

A Büchi automaton [Thomas 1990] is an ω -automaton with a fairness constraint $F \subseteq Q$. A run q of a Büchi automaton is successful iff there is at least one state $p \in F$ such that p appears infinitely often in q . It can be shown that the class of languages accepted by Büchi automata is exactly the class of ω -regular languages, which have the form $\mathcal{L} = \cup_{i=1}^n U_i.V_i^\omega$, where U_i, V_i are regular languages and $V^\omega = \{v_0v_1v_2 \dots \mid v_i \in V, i \geq 0\}$. There are several variations of ω -automata, including Muller, Rabin, Streett, L - and $\forall$ -automata (see Clarke et al. [1990] for a comparison) as well as Edge-Streett-Rabin automata [Hojati et al. 1994], with the essential differences lying in the acceptance condition F . All of the aforementioned automata are equivalent in expressiveness; they all accept precisely the class of ω -regular languages. However, the more expressive fairness constraints of some types of automata allow for a more concise representation. For example, Hojati et al. [1994] have shown that one can construct a class of Edge-Streett automata whose translation into L -automata incurs an exponential increase in size.

We can view an automaton $\mathcal{A}_M$ as a specification of a set of behaviors or traces of a system M if we identify its input alphabet with the externally observable states of M and view its language $\mathcal{L}(\mathcal{A}_M)$ as the set of possible externally observable behaviors of M . Given the automata $\mathcal{A}_{M,I}$ and $\mathcal{A}_{M,S}$ for an implementation and a specification system M_I and M_S , respectively, trace inclusion between M_I and M_S amounts to language containment between the corresponding languages, i.e., $\mathcal{L}(\mathcal{A}_{M,I}) \subseteq \mathcal{L}(\mathcal{A}_{M,S})$.

It can be shown that the expressive power of ω -automata is equivalent to the so-called *monadic second-order theory of one successor* or S1S (see Thomas [1990]). Moreover, ω -regular languages are strictly more expressive than LTL; for example, one can show that the property “predicate q holds in every other state” cannot be expressed in LTL [Wolper 1983], whereas one can easily construct a Büchi-automaton that recognizes the language $\mathcal{L} = (q\Sigma)^\omega$. Wolper [1983] and Vardi and Wolper [1994] propose extensions to LTL where the temporal connectives are ω -automata; they show this class of temporal logics to be equivalent in expressiveness to ω -regular languages.

3. VERIFICATION TECHNIQUES AND TOOLS

Having introduced a number of frameworks for describing systems and asserting their conformance with a specification, we now focus our attention on methods for *verifying* such assertions.

An important aspect is the degree of automation offered by a verification technique; the “ideal” verification tool would, given a system description and a specification, decide within an “acceptably short” amount of time whether or not the specification is met. Unfortunately, creating such a tool is often unrealistic in practice, if not impossible in theory, due to incompleteness results [Duffy 1991]. One can in fact argue that it is unreasonable to always expect completely automatic verification of designs that result from an intellectual process involving highly skilled engineers.

Currently, available techniques range from completely automatic methods for verifying temporal logic specifications (Section 3.1) and language containment (Section 3.2) of finite state transition systems, to interactive theorem-proving using logical calculi (Section 3.3). Finally, Section 3.4 presents some approaches that integrate automatic and deductive approaches.

3.1 Model Checking

In Section 2, we introduced temporal logic as a framework for the specification of the temporal properties of a design. In this section we consider the question of how to determine whether or not a given design satisfies a temporal formula. We present algorithms that efficiently decide this question for finite structures.

For a branching-time logic, a structure $M = (S, R, L)$ is said to be a *model* for a formula f if there exists a state $s \in S$ such that $M, s \models f$. For

a finite structure M (one whose state space S has a finite number of elements), the question whether M is a model for a given formula is decidable. More generally, one can phrase the *model-checking problem* as follows [Emerson 1990]: for a finite structure M and a formula f , label each state $s \in S$ with f iff $M, s \models f$ holds, or equivalently, compute the set $S_f \subseteq S$ such that $M, s \models f$ holds iff $s \in S_f$. The model-checking problem for linear time logic can be phrased similarly in terms of paths. In many applications, a simpler problem statement is sufficient: “Does $M, s \models f$ hold for *all* states $s \in S$ ” for BTL and “Does $M, x \models f$ hold for *all* paths $x \in X$ ” for LTL?

Even though the model-checking problem for finite structures is decidable in principle, there does not necessarily exist a model-checking algorithm with an acceptable time complexity (in terms of the sizes of the state space and the formula) for a given temporal logic. Generally, there is a tradeoff between the efficiency of decision procedures and the expressiveness of the underlying logic. For example, model checking for both propositional linear time logic and for CTL* has been shown to be PSPACE-complete [Sistla and Clarke 1985; Clarke et al. 1986], whereas algorithms with time complexity that is polynomial in the cardinality of S (the model’s state space) exist for CTL. Worse, S tends to be very large in nontrivial applications. For a structure whose states are tuples of k Boolean variables, $|S| = 2^k$, i.e., the size of the state space is exponential in the number of state-holding elements.

In this section, we first introduce the basic concepts behind model checking using a decision procedure for CTL as an example. In verification practice, an important factor is the choice of a “good” representation for sets of states. We present an algorithm based on the symbolic representation of state sets as binary decision diagrams, which are a suitable heuristic for many applications. Finally, we introduce a decision procedure for trajectory assertions over partially ordered state spaces. Section 3.2 presents algorithms for model checking based on language containment.

3.1.1 Explicit State Model Checking. Clarke et al. [1986] present a model-checking algorithm for CTL. A CTL formula f is divided into its subformulas $f_1, \dots, f_k$; and the states of the state-transition graph associated with a structure $M = (S, R, L)$ are labeled with the subformulas that hold for a particular state. Let the size $|f|$ of a formula be one plus the number of its subformulas (i.e., the number of nodes in its parse tree, ignoring parentheses). The algorithm proceeds by successively labeling the states with subformulas of size $i = 1, \dots, |f|$. Subformulas of size one must be atomic propositions; therefore L provides the initial labeling for $i = 1$.

For $i > 1$ and a subformula g of size $|g| = i$, we know that the state graph has already been labeled with the subformulas corresponding to the operands of the outermost operator of g , which must be of size smaller than i .

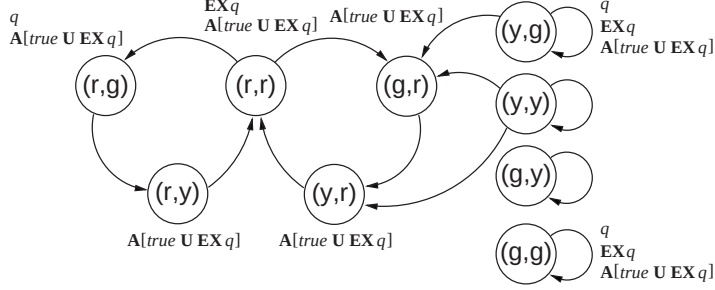


Fig. 4. State transition graph labeled with subformulas.

We can determine if a state s needs to be labeled with g , as follows: In the case $g = \neg g_1$, s is labeled with g if s is not labeled with g_1 ; the case $g = g_1 \wedge g_2$ is treated analogously. For $g = \mathbf{AX}g_1$ ($g = \mathbf{EX}g_1$), s is labeled with g if all (some) successors of g are labeled with g_1 . For $g = \mathbf{E}[g_1 \mathbf{U} g_2]$, first every state that is labeled with g_2 is labeled with g . Second, any state that is labeled with g_1 and has a successor labeled with g is labeled with g . The second step is repeated until no further nodes can be labeled with g . This reachability computation can be performed in $O(|S| + |R|)$ time. A similar approach is used for $g = \mathbf{A}[g_1 \mathbf{U} g_2]$. This yields a CTL model-checking algorithm of time complexity $O(|f|(|S| + |R|))$.

In many applications, one needs to verify that a formula f holds for a subset $S_{init} \subset S$ only, where S_{init} is the set of initial states of the model. In this case, it suffices to consider only the subset of the state space reachable from S_{init} , and the correspondingly restricted transition relation. As the reachable state space is often much smaller than S , this makes explicit model checking practical for larger systems than could be considered otherwise.

As an example, consider Figure 4, which shows the state labeling that results from applying the model-checking algorithm to the traffic-light controller from Section 2.1.1.2 and the CTL formula

$$p = \mathbf{AF}(\mathbf{EX}[ew = g]) = \mathbf{A}[true \mathbf{U} \mathbf{EX}[ew = g]].$$

This formula has the subformulas $true$, $[ew = g]$ and $\mathbf{EX}[ew = g]$. First, all states are labeled trivially with $true$, and all states s with $[ew = g] \in L(s)$ are labeled with $[ew = g]$. In the figure, we use the abbreviation $q = [ew = g]$ and omit the $true$ labels. In the next step, all states that have at least one successor labeled with q are labeled with $\mathbf{EX}q$. This is the case for state (r, r) because of the transition $(r, r) \rightarrow (r, g)$, and for the states (y, g) and (g, g) because of their self-loops. In the third and final iteration, we label states with $p = \mathbf{A}[true \mathbf{U} \mathbf{EX}q]$. A state that is labeled with $\mathbf{EX}q$ can be immediately labeled with p . Furthermore, a state labeled with $true$ (which applies to all states) is labeled with p if all of its

successors satisfy p , which can be determined by depth-first search of the graph. This process is repeated until all states have been examined.

3.1.2 Symbolic Model Checking. The model-checking technique described in the previous section requires that the entire state transition graph be constructed. Thus, the space requirements are at least linear in the size of the model's reachable state-space. However, the latter is often exponential in the number of state-holding elements (e.g., latches) of a design. For instance, a device with only two 32-bit registers would already have ca. 10^{20} states.

An alternative to explicit enumeration of the reachable state space is to use a symbolic representation. Consider a design with n state holding elements (e.g., 1-bit registers). A state assigns a Boolean value to each of these registers and can be represented by the Boolean vector $V = (v_1, \dots, v_n)$. A set G of states can be represented by a Boolean function $G(V)$, also referred to as the *characteristic function* of G , which is *true* iff V represents a state in G . The design's state-transition relation $R \subseteq S \times S$ can be represented symbolically in a similar fashion [Burch et al. 1994]: Let $V' = \{v' \mid v \in V\}$ be the set of *next-state variables*. Then, represent R by its characteristic function $R(V, V')$, such that $R(V, V')$ is *true* iff $(s, s') \in R$ and V and V' have valuations corresponding to states s and s' , respectively.

When using a synchronous design style, state transitions correspond to the functions of the combinational circuitry that maps the current state (i.e., the outputs of the latches/registers) to new values for each state variable. In other words, $v'_i = f_i(V)$. This leads to the next state relation for state variable v_i ,

$$R_i(V, V') = (v'_i = f_i(V)). \quad (7)$$

Combining these, the symbolic representation of the state transition relation for the entire device is

$$R(V, V') = \bigwedge_{i=1}^n R_i(V, V'). \quad (8)$$

Note that this corresponds to a closed, deterministic model. Often the next-state functions f_i also depend on input signals whose values are assigned nondeterministically. These inputs can be modeled by additional variables that are not constrained by R (see Section 2.1.1.1).

In asynchronous designs, each component has conditions under which it is enabled to change the value of its outputs. Aside from these conditions, components operate independently. The next-state relation for an asynchronous design is typically obtained by taking the union of the relations for each component.

3.1.2.1 Symbolic Model-Checking Algorithm. In Section 2.1.4 we showed how the semantics of a temporal logic (the μ -calculus) can be given in terms of sets of states and fixpoints of operations. Symbolic model checking uses the symbolic representation for sets of states from the previous section to “implement” these semantics [Burch et al. 1990]. Operations on sets (e.g., intersection) correspond to Boolean operations on the characteristic functions, while the fixpoint operators are implemented by iterations (which converge because of the model’s finiteness).

In the following, we describe symbolic model checking for CTL [Burch et al. 1994]. A model checker for the μ -calculus can be implemented analogously [Burch et al. 1990]. Let $S[f] \subseteq S$ denote the set of states for which a CTL formula f holds and let $S[f](V)$ denote its characteristic function. We assume that the set of atomic propositions is the set of variables V . Then, for an atomic proposition, $v \in V$, the set of states for which v holds is characterized by $S[v](V) = v$.

For a nonatomic formula f , the characteristic function $S[f](V)$ is computed recursively from the characteristic functions for the subformulas of f . This is straightforward for Boolean operations: For example, for $f = f_1 \wedge f_2$, we have $S[f] = S[f_1] \cap S[f_2]$, and thus $S[f](V) = S[f_1](V) \wedge S[f_2](V)$.

The next-state operators are expressed in terms of quantification over the transition relation³: for $f = \mathbf{AX}f_1$, we have $S[f](V) = F_{\mathbf{AX}}(S[f_1](V))$ with

$$F_{\mathbf{AX}}(Q(V)) = \forall_{v'_1} \dots \forall_{v'_n} R(V, V') \Rightarrow Q(V'). \quad (9)$$

This is precisely Eq. (2) rephrased in terms of characteristic functions.

Similarly, we can use (3) to obtain the characteristic function for $f = \mathbf{AG}f_1$,

$$S[f](V) = \mu Q(V). S[f_1](V) \wedge F_{\mathbf{AX}}(Q(V)).$$

The $\mathbf{A}[\cdot \mathbf{U} \cdot]$ operator can be treated similarly, while the other modalities are expressible in terms of the latter, and the two above, and Boolean operations.

As indicated before, this approach is applicable to the construction of model checkers for the full μ -calculus [Burch et al. 1990], and to CTL^F [Burch et al. 1994]. LTL model checking is also possible, either directly [Burch et al. 1990] or via reduction to CTL^F model checking [Clarke et al. 1994].

An aspect of model checking that is extremely important in practice is the ability to generate counterexamples. Such counterexamples provide traces of incorrect behavior (similar to simulation output), which can help the designer identify the cause of the error. Suppose we attempt to check

³Note that we have chosen to present the cases for the $\mathbf{AX}$ and $\mathbf{AG}$ operators; in Burch et al. [1994], $\mathbf{EX}$, $\mathbf{EG}$, and $\mathbf{E}[\cdot \mathbf{U} \cdot]$ are used to express the universally quantified operators.

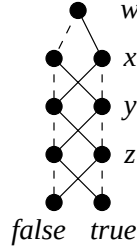


Fig. 5. Binary decision diagram for odd-parity of four variables.

that $M, S_0 \models f$ for some CTL formula f , where S_0 denotes the set of initial states and verification fails. A counterexample is a path of M starting in a state $s \in S_0$ that violates f . For instance, in the case $f = \mathbf{AG}p$, a counterexample can be found essentially by starting with the set $S[\neg p]$, performing backwards reachability analysis until the set S_0 is reached, and picking states from the sets of states arising from the iterative reachability computation. Hojati et al. [1993] describe a method for finding counterexamples for CTL^F formulas.

3.1.2.2 Representation of Boolean Functions. For practical applicability of symbolic model checking, it is crucial that the Boolean characteristic functions involved are represented efficiently. Ordered binary decision diagrams (OBDDs) [Bryant 1986; Bryant 1992] often work well in practice.

A binary decision diagram is a directed acyclic graph whose interior nodes are labeled with the variables of the Boolean function, and which has two leaf nodes labeled *true* and *false*. Each interior node has two outgoing arcs labeled *true* and *false*, respectively, which correspond to the two possible valuations of the node's variable. For a given valuation of the variables, the value of the function is determined by the label of the leaf node reached by a traversal of the graph from the root along arcs whose labels correspond to the valuation.

An *ordered* binary decision diagram (OBDD) has the additional property that on every path between a root node and leaf nodes labeled *true* or *false*, no variable appears more than once, and on all such paths the variables appear in the same order. Subject to a reduction criterion [Bryant 1992; Brace et al. 1990] and for a fixed variable ordering, BDDs⁴ are a canonical representation for Boolean functions, in the sense that the BDDs for two functions f_1 and f_2 are isomorphic iff $f_1 = f_2$.

Common subexpressions are represented by multiple incoming edges to the same vertex. This sharing can make a BDD much more compact than other representations (e.g., sum-of-products form). For example, Figure 3 shows the BDD for the function that computes the parity of four variables. “True” edges are drawn with solid lines and “false” edges are marked with dashed lines. The parity of n variables is represented by an BDD with

⁴In the following, “BDD” refers to a reduced OBDD unless indicated otherwise.

$2n + 1$ nodes, whereas the sum-of-products representation has 2^{n-1} min-terms.

Operations on BDDs can be implemented efficiently: The BDD of a function $f_1 \langle op \rangle f_2$, where $\langle op \rangle$ is a Boolean operation, can be computed from the BDDs of f_1 and f_2 in time linear in the product of the sizes of the two. Other operations such as quantification over a variable can be implemented efficiently as well.

Although BDD operations are logically sound for any variable ordering, the choice of variable ordering can critically affect the size of the BDD for a given function, and thus also the performance of the operations. For example, the size of the BDD for the most significant bit of the sum of two integers represented as bit-vectors is linear in the word size if the bits of the operands are interleaved (essentially, the BDD represents the carry chain of a ripple-carry adder). On the other hand, if all of the bits of one operand word precede the bits of the other word, then the size of the BDD will be exponential in the number of bits in the words. In this case, the BDD must have a separate node for each integer value of the first word to “memorize” the value of this word so it can be added to the other argument [Bryant 1992]. The problem of finding an optimal variable ordering for a given BDD is co-NP complete [Bryant 1986]. In practice, heuristic approaches are used to determine an acceptable ordering. These can be divided into two classes: static ordering heuristics exploit information about the structure of the underlying problem and have been used successfully, e.g., in applications that require building the BDD corresponding to a combinational circuit [Malik et al. 1988; Fujita et al. 1993]. Dynamic reordering, in contrast, is aimed at reducing the number of nodes in an existing BDD. A variety of search heuristics have been proposed [Rudell 1993; Panda and Somenzi 1995].

There are functions such as integer multiplication for which no “good” variable ordering exists [Bryant 1991b]. In fact, any representation of Boolean functions must use exponential space for “most” functions. The success of BDDs is that they work well for many Boolean functions that occur when verifying real designs. Many researchers have explored variants of BDDs to find representations that are more efficient for functions where BDDs are impractical. These include snDDs [Jain et al. 1996]; EVBDDs [Lai and Sastry 1992]; MTBDDs [Clarke et al. 1993]; BMDs [Bryant and Chen 1995]; HDDs [Carke et al. 1995]; and MDGs [Corella et al. 1997]. Bryant [1995] provides a good overview of recent work in this area.

3.1.2.3 Practical Issues in BDD-Based Model Checking. The time complexity of operations on BDDs is polynomial in the size of the BDDs. However, the BDD sizes themselves can be exponential in the number of BDD variables for ill-suited problems. Several approaches that improve the tractability of model-checking problems have been proposed.

For many designs, the size of the BDD for the state transition relation R exceeds available resources. However, there are alternative approaches to

image computations such as Eq. (9), which do not require a monolithic BDD for R to be constructed. Burch et al. [1994] describe approaches where R is conjunctively or disjunctively partitioned and represented by the list of the BDDs for the partitions. Quantification is performed on the partitions as early as possible, reducing the size of intermediate BDDs. For synchronous designs, images can be computed from the vector of the next-state functions $(f_1, \dots, f_n)$ without use of a relational representation [Coudert and Madre 1990]. Hu et al. [1992] recursively decompose the image computation problem based on the structure of the program expressions describing the system. Lee et al. [1994a] verify safety properties by successively strengthening an invariant of an asynchronous program, considering the transition relations of its components one by one.

Another area of improvement is the representation size of the characteristic functions for the intermediate sets of states arising from iterative fixpoint calculations. Hu et al. [1994] apply the idea of using implicitly conjoined BDDs to the characteristic functions of these sets of states. The decomposition is noncanonical; the approach employs heuristics to recombine some of the BDDs to prevent the list from growing excessively and applies a simplification procedure to pairs of BDDs which is based on the observation that one BDD defines a “don’t care” set for a second BDD with which it is conjoined. McMillan [1996] proposes a conjunctively decomposed representation that also provides canonicity. Ravi and Somenzi [1995] reduce the size of intermediate BDDs by replacing them with smaller BDDs of dense subsets, which leads to state space traversal in a combination of breadth-first and depth-first exploration. Another technique is based on the elimination of state variables that can be expressed as functions of other state variables [Hu and Dill 1993]. During the fixpoint calculation, the functional dependencies are substituted for the eliminated variables.

3.1.3 Refinement and Model Checking. Recall from Section 2.2.4 that verifying refinement between an implementation M_I and a specification M_S generally requires showing that for every observable behavior of M_I , there exists a corresponding observable behavior of M_S . In many applications, this condition is verified using deductive methods (described in Section 3.3). Under certain conditions, refinement can be expressed as a safety property, which permits treatment with automatic model checking if the systems have finite state.

Refinement can be formulated as a safety property if the specification behavior corresponding to a given implementation behavior can be obtained through a state-by-state mapping. Assume the implementation and specification are given as transition systems $M_I = (S_I, S_{0,I}, R_I)$ and $M_S = (S_S, S_{0,S}, R_S)$. Assume further that there exists an abstraction function (refinement mapping) $\mathcal{A} : S_I \rightarrow S_S$, such that the specification trace p_S corresponding to an implementation trace $p_I = s_0 s_1 s_2 \dots$ is given by $p_S = \mathcal{A}(s_0) \mathcal{A}(s_1) \mathcal{A}(s_2) \dots$. Then, M_I is a refinement of M_S , $M_I \sqsubseteq M_S$ iff

$$\mathcal{A}(S_I) \subseteq S_M \quad (10)$$

and

$$\forall_{s, s' \in S_I} (s, s') \in R_I \Rightarrow ((\mathcal{A}(s) = \mathcal{A}(s') \vee (\mathcal{A}(s), \mathcal{A}(s')) \in R_S). \quad (11)$$

Condition (11) is satisfied iff

$$\mathcal{R}(s) = \forall_{s' \in S_I} (s, s') \in R_I \Rightarrow ((\mathcal{A}(s) = \mathcal{A}(s') \vee (\mathcal{A}(s), \mathcal{A}(s')) \in R_S)$$

is a safety property of the implementation M_I . Lee et al. [1994b] use safety properties of this form to verify refinement between programs written in ST [Staunstrup 1994], a UNITY-like guarded command language. In this case, the transition relation is given as the union of transition relations corresponding to the individual guarded multiassignments. The refinement predicate has the form of a product of clauses, one for each guarded command of the implementation, and each of which is a sum of clauses for the specification commands. The BDD for $\mathcal{R}$ is built directly from the program texts for M_I and M_S and the expression for the abstraction mapping, without explicit construction of the transition relation.

For a given notion of refinement, the following is an important question: Assuming $M_I \sqsubseteq M_S$ and a temporal formula φ that holds for M_S , $M_S \models \varphi$, does φ also hold for M_I , i.e., does refinement preserve the truth of φ ? If φ is a simple safety property, as introduced in Section 2.1.5, this is indeed the case, and can be shown by a simple induction argument. Using a slightly different definition of refinement, Clarke et al. [1994] show that properties stated in $\forall\text{CTL}^*$ (CTL^* restricted to universal path quantifiers only) are preserved as well. Furthermore, this restriction permits compositional verification under an assume-guarantee paradigm [Grumberg and Long 1994]. This allows properties of a system to be deduced from properties of its components, thus avoiding model checking of the complete model.

Abadi and Lamport [1991] consider refinement in the abstract framework described in Section 2.2.4. They provide conditions on M_I and M_S that ensure that if M_I implements M_S in the sense of (5), there exists an abstraction function. In Abadi and Lamport [1993], a compositional theory is developed for the same general framework.

3.1.4 Symbolic Trajectory Evaluation. In Section 2.1.6, we introduced trajectory formulas (TF), which form a temporal logic of restricted expressiveness. Seger and Bryant [1995] describe an efficient model-checking approach for TF called symbolic trajectory evaluation (STE). STE uses a lattice representation of circuit states; we consider the case where nodes have quaternary values represented by the lattice $\mathcal{T} = \{X, 0, 1, \top\}$. Here, 0 and 1 denote low and high voltages, respectively, X denotes an unknown

or indeterminate value, and $\top$ represents an over-constrained value.⁵ States of a circuit with n nodes are represented by values in $\mathcal{T}^n$. Seger and Bryant [1995] develop the underlying theory for arbitrary lattices; for simplicity, we restrict our presentation to the lattice $\mathcal{T}^n$.

Values on $\mathcal{T}$ are partially ordered: $X \sqsubseteq 0, 1 \sqsubseteq \top$. This ordering is an equivalence relation: for all values $a, b \in \mathcal{T}$, $a \sqsubseteq a$, and $a \sqsubseteq b \wedge b \sqsubseteq a$ iff $a = b$. The partial order for $\mathcal{T}^n$ is the element-wise extension of the order for $\mathcal{T}$: For $\alpha, \beta \in \mathcal{T}^n$, $\alpha \sqsubseteq \beta$ iff $\alpha_i \sqsubseteq \beta_i$ for each component α_i of α and the corresponding component β_i of β . For any two elements of $\mathcal{T}^n$ there is a unique least upper bound: $\text{lub}(\alpha, \beta) = \gamma$ iff $\alpha \sqsubseteq \gamma$, $\beta \sqsubseteq \gamma$, and for any δ with $\alpha \sqsubseteq \delta$ and $\beta \sqsubseteq \delta$, $\gamma \sqsubseteq \delta$. We extend the partial order $\sqsubseteq$ element-wise to sequences over $\mathcal{T}^n$ in the natural way.

Executions of the circuit model are represented by sequences of elements of $\mathcal{T}^n$. Let g be such a sequence. The fundamental observation behind STE is that for a given TF f , there exists a unique *defining sequence* δ_f such that g satisfies f iff $\delta_f \sqsubseteq g$.

We first construct the defining sequence δ_f for formulas f without symbolic variables. In the following, let α^ω denote the infinite sequence $\alpha, \alpha, \dots$.

Simple predicates: $\delta_{a_i \text{ is } v} = U_{i, v}, X_n^\omega$

Here, $U_{i, v}$ is the lattice element whose i^{th} element is v and all other elements have value X , and X_n denotes the lattice element $(X, \dots, X)$.

Conjunction: $\delta_{f \wedge g} = \text{lub}(\delta_f, \delta_g)$

A sequence that satisfies $f \wedge g$ must satisfy the constraints of both f and g . If f specifies that a node must be 0 on a time step when g specifies a value of 1 for the same node, then $f \wedge g$ will specify a value of $\top$ for that node on that time step. This indicates that f and g are *inconsistent* and can be flagged as a probable error in the specification.

Next time: $\delta_{Nf} = X_n, \delta_f$

Intuitively, Nf asserts nothing about the head of the defining sequence and makes the assertions of f about the tail of the sequence.

Domain restriction: $\delta_{f \text{ when } 0} = X_n^\omega$, and $\delta_{f \text{ when } 1} = \delta_f$.

In words, $f \text{ when } 1$ is equivalent to f , and $f \text{ when } 0$ gives an unrestricted defining sequence.

Two observations are in order. First, conjunction has a natural interpretation as a least upper bound of lattice elements. Disjunction, on the other hand, does not correspond to the greatest lower bound, which is why disjunction and negation are not allowed in trajectory formulas. Second,

⁵ This model is used in current versions of the Voss verification tool [Seger 1993; Hazelhurst and Seger 1995]. The previously used ternary model is described in Seger and Bryant [1995].

even though defining sequences are infinite, after a finite prefix, every element of the sequence is X_n . The length of this prefix is one greater than the maximum nesting of next time operators in the trajectory formula.

Symbolic trajectory formulas can be handled by representing each node, nd , with a pair of two symbolic Boolean formulas (nd_0, nd_1), using valuations of $(0,0)$, $(1,0)$, $(0,1)$, $(1,1)$ to encode X , 0 , 1 , $\top$, respectively. Operations on $\mathcal{T}$ such as conjunction, lub, etc., translate into Boolean operations on the elements of such pairs of symbolic Boolean formulas that can be performed using BDDs (see Section 3.1.2.2).

Verifying a trajectory assertion $M \models [A \Rightarrow C]$ requires showing that each sequence that satisfies A and is consistent with the circuit model M also satisfies C . Similar to defining sequences, a *defining trajectory* is used to characterize the set of sequences that are both possible behaviors of the model and also satisfy a given trajectory formula.

The circuit's behavior is given as a function, $Y : \mathcal{T}^n \rightarrow \mathcal{T}^n$. This function must be monotonic: if $\alpha \sqsubseteq \beta$ then $Y(\alpha) \sqsubseteq Y(\beta)$. Intuitively, $Y(\alpha)$ may have X values if α has X values. If β can be obtained from α by replacing some X values with 0 or 1 , then this must not result in more X values in $Y(\beta)$ than were present in $Y(\alpha)$. In practice, Y can be obtained implicitly through symbolic simulation on a circuit model. The main distinction between a symbolic simulator and a conventional logic simulator is that it allows variables to appear in the inputs. The values of the circuit's internal nodes and outputs then become functions of these variables (see Bryant [1991a]). With STE, the variables have domain $\mathcal{T}$; the values of nodes and outputs can be represented by pairs of BDDs, as indicated above.

Given a defining sequence δ_A , the corresponding defining trajectory is

$$\tau_A^j = \begin{cases} \delta_A^0 & \text{for } i = 0, \\ \text{lub}(\delta_A^i, Y(\tau_A^{j-1})) & \text{for } i > 0. \end{cases} \quad (12)$$

The i^{th} element of τ_A corresponds to the least specified state that the circuit can reach at step i given that A holds on the prefix of the execution up to state i .

We can now state the central theorem of STE: For a model structure M and trajectory formulas A, C ,

$$M \models [A \Rightarrow C] \Leftrightarrow \delta_C \sqsubseteq \tau_A.$$

This theorem implies that to verify a trajectory assertion, it is sufficient to compute the sequences τ_A and δ_C and compare them with respect to the partial order $\sqsubseteq$. The sequences δ_A and δ_C can be computed recursively, as described above; τ_A is computed from δ_A according to (16), requiring a single symbolic simulation. As noted above, every element of δ_C is X_n after a finite prefix; therefore it is sufficient to compute and compare only these prefixes of τ_A and δ_C .

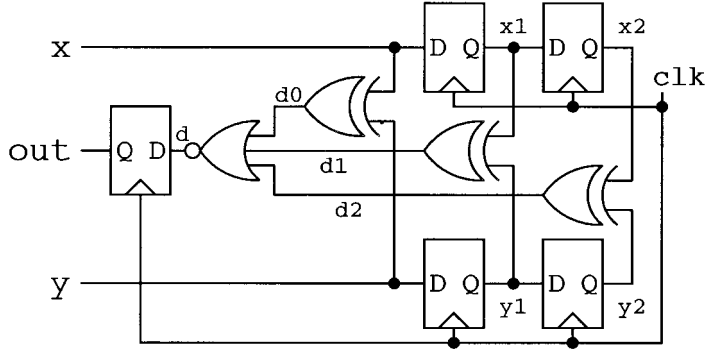


Fig. 6. Implementation of a simple correlator.

There are two serious limitations in the STE framework: negation is not permitted in trajectory formulas and the next-state function is essentially restricted to be deterministic. Addressing the first restriction, Hazelhurst [1996] has extended STE to a richer logic supporting negation and an “until” operator; however, efficient model checking is not always possible for this logic. Weih and Greenstreet [1996] have developed a framework that allows the application of STE to certain classes of speed-independent circuits.

3.1.4.1 An Example. Consider the correlator specified in Eq. (4). Figure 6 shows an implementation of this correlator. A state of this circuit can be represented by the tuple $(x, y, out, x1, y1, x2, y2)$. Note that $d0, d1, d2$, and d are combinational functions of the other nodes. Let A denote the antecedent of the first assertion from Eq. (4):

$$A = (x \text{ is } a) \wedge (y \text{ is } a) \wedge (\mathbf{N}(x \text{ is } b) \wedge \mathbf{N}(y \text{ is } b)) \wedge (\mathbf{N}^2(x \text{ is } c) \wedge \mathbf{N}^2(y \text{ is } c)).$$

The consequent, C , is $\mathbf{N}^3(out \text{ is } 1)$. The defining sequence for C , δ_C is

$$0 : ((F, F), (F, F), (F, F), (F, F), (F, F), (F, F), (F, F))$$

$$1 : ((F, F), (F, F), (F, F), (F, F), (F, F), (F, F), (F, F))$$

$$2 : ((F, F), (F, F), (F, F), (F, F), (F, F), (F, F), (F, F))$$

$$3 : ((F, F), (F, F), (F, T), (F, F), (F, F), (F, F), (F, F))$$

Note that we only need to consider the first four elements of the defining sequence, as all others describe completely unconstrained states. Values in $\mathcal{T}$ are encoded as pairs of Boolean expressions as defined above. This asserts $(out \text{ is } 1)$ at step 3. The defining sequence for A , δ_A is:

$$0 : ((\neg a, a), (\neg a, a), (F, F), (F, F), (F, F), (F, F), (F, F))$$

$$1 : ((\neg b, b), (\neg b, b), (F, F), (F, F), (F, F), (F, F), (F, F))$$

$$2 : ((\neg c, c), (\neg c, c), (F, F), (F, F), (F, F), (F, F), (F, F))$$

$$3 : ((F, F), (F, F), (F, F), (F, F), (F, F), (F, F), (F, F))$$

This asserts that x and y both have value a on step 0, both have b on step 1, and both have c on step 2. Their values are unconstrained on step 3. Again, we only need to consider the first four elements of the defining sequence. To compute the defining trajectory for A , we need the next state function Y , for which we assume the obvious interpretation of Figure 6. This yields the defining trajectory of A , τ_A :

$$0 : ((\neg a, a), (\neg a, a), (F, F), (F, F), (F, F), (F, F), (F, F))$$

$$1 : ((\neg b, b), (\neg b, b), (F, F), (\neg a, a), (\neg a, a), (F, F), (F, F))$$

$$2 : ((\neg c, c), (\neg c, c), (F, F), (\neg b, b), (\neg b, b), (\neg a, a), (\neg a, a))$$

$$3 : ((F, F), (F, F), (F, T), (\neg c, c), (\neg c, c), (\neg b, b), (\neg b, b))$$

It is straightforward to show that $\delta_C \sqsubseteq \tau_A$, verifying the first assertion of Eq. (4). The second assertion can be verified in a similar manner.

3.1.5 Model-Checking Tools. In the following, we briefly introduce a selection of tools, both of academic origin and commercially marketed, which implement the methods presented in the previous sections.

EMC (extended model checker) was one of the first model checkers to be implemented [Clarke et al. 1986]. It constructs an explicit representation of the state graph from a program written in a subset of CSP and supports model checking of formulas in CTL and CTL^F.

Mur ϕ [Dill et al. 1992; Dill 1996] is both a UNITY-like description language and an explicit state model-checking system. Specifications in Mur ϕ are given as simple safety properties, which are verified by explicit state space traversal. Extensions to the Mur ϕ system exploit techniques to reduce the size of the representation of the reachable state space [Ip and Dill 1996a; Ip and Dill 1996b]. Ever [Hu et al. 1992] is a BDD-based symbolic model checker that can act as a back-end for Mur ϕ ; Ever is suitable as a stand-alone tool as well. The Ever language supports high-level constructs such as array and record types, as well as imperative statements such as sequential composition.

SMV [McMillan 1992; Dill 1996] is probably the most widely used symbolic model checker to date. System descriptions in the SMV language are given in terms of a set of equations that determine the next-state

relation; programs may be structured into parameterized modules. SMV model checks specifications given in CTL and CTL^F. A recent version supports word-level model checking [Clarke et al. 1996] by using hybrid decision diagrams (HDDs) [Clarke et al. 1995] as the underlying data structure, which permits model checking of properties involving words, i.e., bit vectors interpreted as integers.

CVE [Bormann et al. 1995] is an industrial verification environment developed at Siemens. It supports model checking of designs described in VHDL or EDIF against specifications given in a temporal logic called CIL, which is equivalent to a subset of CTL. If an error is detected, CVE can generate a VHDL testbench that exposes the fault. In addition, CVE provides combinational and sequential equivalence checking ([Filkorn 1991]; see Section 3.2.2). Checkoff-M [Xu et al. 1997], by Abstract Inc., is a commercially marketed product based on CVE, which includes Verilog support as well.

RuleBase [Beer et al. 1996], developed at IBM, is an industry-oriented model-checking tool built on SMV that provides a graphical user interface, a temporal logic defined on top of CTL, support for VHDL and Verilog, and debugging support. VIS [Brayton et al. 1996] integrates model checking with other verification techniques such as combinational and sequential equivalence checking. VIS accepts design descriptions in a synthesizable subset of Verilog, and supports CTL^F model checking. Interaction with the SIS synthesis tool [Sentovich et al. 1992] is provided through a common intermediate format.

SPIN [Holzmann 1997] is a model checker targeted at the high-level verification of distributed systems. SPIN accepts model descriptions in the specification language PROMELA, which provides high-level constructs such as communication channels. Specifications are given in LTL and verified by a model-checking algorithm based on the automata-theoretic approach described in Section 3.2.3

Voss [Seger 1992] is a verification system based on symbolic trajectory evaluation. Trajectories are evaluated using a symbolic simulator that can operate either on a switch-level model [Bryant et al. 1987] of a transistor netlist, or a state machine extracted from a gate-level netlist or a description in a subset of VHDL. Voss provides a metalanguage front-end in the form of `fl`, an ML-like but fully lazy functional language.

The FormalCheck tool [Kurshan 1997], marketed by Bell Labs Design Automation, uses COSPAN as its verification engine, which is based on language containment as its underlying formalism (described in Section 3.2.1). Recently, additional commercially available tools for model checking and equivalence checking have been brought to market; see [DAC Exhibits 1997] for a list of products).

3.2 Automata-Theoretic Approaches

This section describes two aspects of automata-theoretic approaches for formal verification. The first is concerned with the verification of contain-

ment between languages recognized by automata on infinite words. Here, automata are mainly a tool to capture the properties of the languages in question. The second aspect originates from the observation that deterministic finite state automata are a natural model for sequential circuits. In this context, the verification goal is usually to show observational equivalence between two automata.

3.2.1 Automata on Infinite Objects and Language Containment. Recall that the language-containment problem involves verifying that the language recognized by an ω -automaton $\mathcal{X}$ is a subset of the language recognized by a second ω -automaton $\mathcal{Y}$, i.e., $\mathcal{L}(\mathcal{X}) \subseteq \mathcal{L}(\mathcal{Y})$ (see Section 2.2.5).

A decision procedure for the language-containment problem can be derived by recasting containment into the language-emptiness problem. Given finite automata $\mathcal{X}$ and $\mathcal{Y}$ with input alphabet Σ , a product automaton, $\mathcal{U}$, with $\mathcal{L}(\mathcal{U}) = \mathcal{L}(\mathcal{X}) \cap (\Sigma^\omega - \mathcal{L}(\mathcal{Y}))$ can be constructed (see e.g., Thomas [1990]). By this construction, $\mathcal{L}(\mathcal{X}) \subseteq \mathcal{L}(\mathcal{Y})$ iff $\mathcal{L}(\mathcal{U}) = \emptyset$. Using Büchi acceptance criteria, automaton $\mathcal{U}$ accepts some word iff there exists a state $s \in F$ reachable from its initial state $\bar{q}_{\mathcal{U}}$ and is reachable from itself. Because the set of states of $\mathcal{U}$ is finite, the language emptiness problem can be decided by reachability analysis of $\mathcal{U}$'s state graph.

Clarke et al. [1990] propose an elegant way to express the language-emptiness problem as a model-checking problem, which they use to obtain decision procedures for language containment between different types of ω -automata. The underlying idea is to construct a temporal structure $M = (S, R, L)$ from a given ω -automaton $\mathcal{X} = (\Sigma, Q, \bar{q}, \Delta, F)$, with $S = Q$, $L(s) = \{s\}$, and $R = \{(s, s') \mid \exists \alpha \in \Sigma (s, \alpha, s') \in \Delta\}$, i.e., M has the same state space as $\mathcal{X}$, and has transitions where $\mathcal{X}$ has a transition enabled by at least one input symbol. The acceptance condition F is translated into a CTL* formula Φ_F such that M is a model of Φ_F iff $\mathcal{X}$ accepts at least one word. For Büchi acceptance, language emptiness can be expressed in CTL*, as shown below:

$$\mathcal{L}(\mathcal{X}) = \emptyset \Leftrightarrow M, \bar{q} \models \neg \mathbf{E}[\mathbf{GF}(\bigvee_{s \in F} s)].$$

Thus the methodologies developed for model-checking problems can be applied to language-emptiness as well.

Kurshan [1989] defines L -automata and L -processes. The main difference to other ω -automata is that their transitions are labeled with elements of a Boolean algebra L , rather than simple elements of an input alphabet. A transition is enabled if the current input symbol is contained in the set of symbols of its label. L -processes are defined in a way that facilitates the modeling of coordination of processes as a *tensor-product*. For the details of these definitions, the reader is referred to Kurshan [1989].

Let $\mathcal{X}$ be an L -automaton, a specification, and let $\mathcal{Y}$ be an L -process, a proposed implementation of $\mathcal{X}$. Verification requires showing that the language of the implementation, $\mathcal{L}(\mathcal{Y})$, is contained in the language of the specification, $\mathcal{L}(\mathcal{X})$. In cases where direct verification is not feasible, it may be possible to find simpler automata $\mathcal{X}'$ and $\mathcal{Y}'$ such that showing $\mathcal{L}(\mathcal{Y}') \subseteq \mathcal{L}(\mathcal{X}')$ establishes $\mathcal{L}(\mathcal{Y}) \subseteq \mathcal{L}(\mathcal{X})$ as well. Kurshan formalizes the relationships that must hold between $\mathcal{X}$, $\mathcal{X}'$ and $\mathcal{Y}$, $\mathcal{Y}'$ using homomorphisms between the languages and the underlying Boolean algebras. This provides a formal basis for simplifying abstractions that allow larger systems to be verified without resorting to unverified assumptions. These techniques are implemented in the COSPAN verification tool [Hardin et al. 1996], which also forms the basis for the commercially available FormalCheck tool [Kurshan 1997].

3.2.2 State Machine Equivalence. A special case of the verification condition “device X' implements specification X ” arises when both X and X' are deterministic and indeed exhibit *identical* behaviors when operated under the same environment behavior. For simplicity, we consider Mealy machines where internal states, inputs, and outputs are represented by Boolean vectors. Formally, such a machine is given by a tuple $X = (\Sigma, \Omega, S, s_0, \delta, \lambda)$, where $\Sigma = \mathcal{B}^l$ denotes the input alphabet; $\Omega = \mathcal{B}^m$ the output alphabet; $S = \mathcal{B}^n$ the set of states; $s_0 \in S$ the initial state; $\delta : S \times \Sigma \rightarrow S$ the state transition function; and $\lambda : S \times \Sigma \rightarrow \Omega$ the output function. In a realization of X as a sequential circuit, the state space S corresponds to the circuit’s latches; Σ and Ω correspond to the circuit’s input and output wires; and δ and λ are realized by combinational circuits.

Two machines X and X' with $\Sigma = \Sigma'$ and $\Omega = \Omega'$ are considered equivalent iff for each input sequence $(\sigma_1, \sigma_2, \dots) \in \Sigma^*$, both machines generate the same output sequence $(\omega_1, \omega_2, \dots) \in \Omega^*$. A notion of equivalence that also relates the generated sequences of states can be accommodated by extending the output function such that it “forwards” the relevant information about the internal state to the outputs.

The problem of showing that two sequential machines are equivalent can be reduced to the problem of finding the reachable state set of a product machine. To show that machine X and X' are equivalent, one can construct the product machine $M^x = (\Sigma \times \Sigma', \mathcal{B}, S \times S', (s_0, s'_0), \delta^x, \lambda^x)$, where $\delta^x((s, s'), \sigma) = (\delta(s, \sigma), \delta'(s', \sigma))$ and $\lambda^x((s, s'), \sigma) = (\lambda(s, \sigma), \lambda'(s', \sigma))$ for all states $(s, s') \in S \times S'$ and input symbols $\sigma \in \Sigma \times \Sigma'$. Intuitively, M^x is a machine that runs X and X' in lock-step on the same inputs and whose output is 1 iff the outputs of X and X' agree. To show that X and X' are equivalent, it is sufficient to compute the set of reachable states Q of M^x . Then, one can verify that M^x produces output 1 for all states $s \in Q$, and for all possible input vectors [Coudert et al. 1989].

The reachable state set of a sequential machine can be computed using a fixpoint calculation similar to the one in the model-checking algorithm in

Section 3.1.2. The main difference is that the next-state relation derived from the next-state function (cf., Eq. (8)) now also depends on the current input, which is left unconstrained in the reachability computation. Touati et al. [1990] present techniques to increase the efficiency of the reachable state set computation. Coudert et al. [1989] present an algorithm for state machine equivalence where they convert the computation of a set's image under the next-state function to a computation of the range of a function derived from the original next-state function. Pixley et al. [1994] use similar techniques to find synchronizing sequences for sequential circuits.

The number of iterations required to compute the reachable state set can be linear in the number of states for circuits that have “long” cycles of states, such as counters. Based on this observation, Filkorn [1991] describes an approach that uses backward traversal of the product machine's state space, which may lead to much faster convergence of the fixpoint calculation. Starting with the set of states of M^x in which the outputs of X and X' differ for at least one input symbol, the set of *nonequivalent* states that can reach such a state is computed. Then, X and X' are equivalent iff the initial state of M^x is not a nonequivalent state.

3.2.2.1 Combinational Equivalence. For two machines X and X' whose state space and initial states are identical, it is a sufficient condition for their equivalence that $\delta = \delta'$ and $\lambda = \lambda'$. Thus, given two sequential circuits using the same state encoding, their equivalence can be established by showing combinational equivalence for the realizations of their next-state and output functions [Bryant 1992].

One of the earliest industrial-scale equivalence checker is the PRIAM tool [Madre and Billon 1988]. The next-state and output functions of both machines are specified in a procedural HDL. From this description, the Boolean functions for the state components and output signals are extracted via symbolic execution; BDDs are used as their representation. Using the above observation, equivalence between the machines is established by comparing these functions. If the comparison fails, PRIAM's diagnosis facility can, under a single fault assumption, pinpoint gates that may be responsible for the fault and suggest modifications to those gates that would rectify the error [Madre et al. 1989].

Verity [Kuehlmann et al. 1995] is a verification system aimed at checking equivalence between a RTL specification and a gate- or transistor-level implementation for large, hierarchically structured designs. Both specification and implementation are described as a hierarchy of macro (cell) instances. Leaf nodes in the hierarchy are verified directly via combinational equivalence checking as described above. Boundary conditions between macros are asserted in the form of constraints defining sets of allowed patterns on their input and output signals. Supermacros that are too large for direct verification are treated by abstracting away (some of) their submacros. Connections between the supermacro and the submacros are treated as inputs and outputs of the supermacro during equivalence

checking. To verify the consistency of the macro interconnection, it is checked that the input/output constraints on the submacro boundaries are maintained. For the verification of leaf macros, a functional extractor computes Boolean functions (represented as BDDs) for the next-state and output functions of the implementation circuit, which are then compared with the corresponding functions obtained from the RTL specification. The extraction scheme handles arbitrary combinations of gate-level and transistor-level models. Verity also provides a counterexample and diagnosis facility.

3.2.3 Language Containment and Model Checking. There is a close connection between ω -automata and model checking. In Section 3.2.1, we saw that the language-containment problem for two ω -automata can be reduced to a model-checking problem. We now point out that the converse holds as well.

The basis of this approach to model checking is to view a temporal formula as an acceptor of infinite structures, and a temporal structure as a generator thereof [Vardi and Wolper 1986a]. For brevity, we describe the approach for linear time logic only.

An LTL formula φ over a set of atomic propositions $\mathcal{P}$ can be understood as an acceptor of infinite words over the alphabet $\Sigma = 2^{\mathcal{P}}$, in the sense that φ accepts a word $x \in \Sigma^\omega$ iff x is a model of φ . Thus a formula φ gives rise to a language $\mathcal{L}_\varphi$ containing precisely the words accepted by φ . It can be shown that one can construct an ω -automaton $\mathcal{A}_\varphi$ from a formula φ in LTL such that $\mathcal{A}_\varphi$ accepts exactly the language defined by φ , i.e., $\mathcal{L}(\mathcal{A}_\varphi) = \mathcal{L}_\varphi$ [Manna and Wolper 1984; Kesten et al. 1993; Vardi and Wolper 1994].

On the other hand, a (finite) temporal structure $M = (S, R, L)$ can be interpreted as a generator of ω -words over $2^{\mathcal{P}}$; the language generated is the set

$$\mathcal{L}(M) = \{L(s_0)L(s_1)L(s_2)\dots \mid s_0, s_1, \dots \text{ is a path of } M\}.$$

M can be straightforwardly extended into a Büchi automaton $\mathcal{A}_M$ that accepts exactly $\mathcal{L}(M)$ by labeling the transitions $(s, t) \in R$ with the atomic propositions $L(s)$. Using these observations, we can now rephrase the model-checking problem as a language-containment problem [Vardi and Wolper 1979a]: $M \models \varphi$ iff $\mathcal{L}(\mathcal{A}_M) \subseteq \mathcal{L}(\mathcal{A}_\varphi)$, or in other words, every sequence generated by M is accepted by φ in the above sense.

In the automata-theoretic framework, model checking can be extended to infinite state spaces by constructing the product automaton for the language-containment test incrementally [Manna et al. 1994]. This results in a semi-decision procedure, i.e., the model checker may either terminate with a positive answer or a counterexample, or not terminate at all.

The corresponding automata-theoretic approaches for branching time logics require the use of automata on infinite trees, rather than words, to

reflect the branching structure of time (see Vardi and Wolper [1986b]; Emerson [1990]; Bernholtz et al. [1994]).

3.3 Deductive Methods

As we saw in Section 2.2.2, specifications and verification conditions can be expressed in general-purpose logic. Verifying that an implementation meets its specification in such a framework is equivalent to proving a theorem in the underlying logic. In principle, this proof could be carried out manually. However, proofs of such theorems are often long and rather tedious in practice, making it likely that they contain errors. Using a mechanized theorem-proving system can ensure soundness and reduce tedium by automating parts of the proof.

3.3.1 Automated Theorem Proving. A theorem-proving program mechanizes a proof system. A *calculus* or *proof system* $\mathcal{C}$ for a logic $\mathcal{L}$ consists of a set of *axioms* $\mathcal{A}$ and a set of *inference rules* $\mathcal{R}$. The axioms are formulas of $\mathcal{L}$ and are usually “elementary” in the sense that they capture the basic properties of the logic’s operators. The general form of an inference rule is

$$\frac{\alpha_1, \dots, \alpha_k}{\beta}$$

The formulas $\alpha_1, \dots, \alpha_k$ are called the *premises* of the rule while β is called the *conclusion*. For example, the formulas $p \Rightarrow (p \vee q)$ and $(p \wedge q) \Rightarrow q$ are axioms and

$$\frac{p, p \Rightarrow q}{q}$$

is an inference rule of the “classical” propositional calculus [Kleene 1967]. Both the axioms and the inference rule are actually axiom and rule *schemas*, i.e., they give rise to an infinite number of axioms and rules obtained by replacing p and q with arbitrary formulas.

A *deduction* in $\mathcal{C}$ of a formula φ_n from a set of assumptions Γ is a finite sequence of formulas $\varphi_1, \dots, \varphi_n$ such that for all $i = 1, \dots, n$, either $\varphi_i \in \mathcal{A} \cup \Gamma$, or φ_i can be obtained by applying an inference rule in $\mathcal{R}$ to some subset of $\{\varphi_1, \dots, \varphi_{i-1}\}$. We say that a formula φ is a *theorem* of Γ , written $\Gamma \vdash \varphi$, if there exists a deduction of φ from Γ . A calculus is said to be *sound* if for all sets of formulas Γ , all theorems of Γ are logical consequences of Γ , and *complete* if all logical consequences of Γ are also theorems (i.e., can be proven in $\mathcal{C}$). For a more complete treatment, see e.g., Duffy [1991].

The discussion so far pertains to rigorous manual proofs as much as to automated systems. However, there are three aspects particular to mechanized proof systems:

- (1) Such a system can mechanically *check* a proof, i.e., verify that a given sequence of formulas $\varphi_1, \dots, \varphi_n$ is indeed a deduction. This is usually not a very difficult task; it suffices to syntactically match the inference rules against the appropriate premises in the sequence and verify that each φ_i is indeed obtained by application of some rule.
- (2) It can assist in the *construction* of a proof. Given a set of assumptions Γ and a goal β , heuristic search techniques may be able to find a deduction such that $\beta = \varphi_n$. In general, this will not succeed completely automatically for “difficult” theorems, but will require a certain amount of human guidance to construct a deduction.
- (3) It permits the use of *decision procedures*. A decision procedure is an algorithm that decides the validity of a class of formulas. For example, BDDs (see Section 3.1.2.2) provide a practical decision procedure for propositional logic. Other decision procedures exist for Presburger arithmetic (essentially integer arithmetic with multiplication by constants only) [Shostak 1979] and even real algebra [Tarski 1951; Harrison 1993]. There are also algorithms for combinations of theories such as linear arithmetic, equality with uninterpreted functions, and arrays under *store* and *select* [Nelson and Oppen 1979; Shostak 1984; Cyruluk et al. 1996].

One can view a decision procedure as giving rise to a class of axioms similar to an axiom schema, except that the class is generally much larger. For example, the axiom schema $p \Rightarrow (p \vee q)$ just generates axioms of this particular form, while a propositional decision procedure gives rise to all propositional tautologies. Note that a search procedure that is complete (i.e., guaranteed to find a deduction) for a class of formulas constitutes a decision procedure for this class. For efficiency, however, practical decision procedures do not in general produce deductions, but algorithmically decide the validity of a formula without referring to the calculus (e.g., BDDs for propositional formulas).

Decision procedures are important in practice, since they can handle many “uninteresting” cases automatically, and thus help to alleviate the tedium of proofs.

A myriad of theorem-proving systems have been implemented, and many have been used for hardware verification, including HOL [Gordon 1988]; ISABELLE [Paulson 1994]; LP [Garland and Gutttag 1989]; Nqthm and ACL2 [Boyer and Moore 1979; Kaufmann and Moore 1996]; Nuprl [Jackson 1992]; OTTER [McCune 1994]; PVS [Owre et al. 1992]; RRL [Kapur and Zhang 1988]; and TLP [Engberg et al. 1992]. These systems are distinguished by, among other aspects, the supported proof style, the underlying mathematical logic, the way automatic-decision procedures are integrated into the system, and the user interface. Proof styles are often characterized as “forward” or “backward.” A forward proof starts with the axioms and assumptions; inferences are then applied until the desired theorem is

proven. A backward proof starts with the theorem as a goal and applies the inverses of inference rules to reduce the theorem to simpler intermediate goals. Sufficiently simple goals are discharged by matching axioms or assumptions or by applying built-in decision procedures.

General-purpose theorem provers are usually based on some form of first- or higher-order logic. In a first-order logic, variables (e.g., Booleans and integers) are distinguished from functions (e.g., a mapping from integers to Booleans). Furthermore, quantification is allowed over variables but not over functions. Higher-order logic also allows quantification over functions. The choice of a logic influences the logical idioms used to write specifications and to describe designs, as well as influencing the proof techniques and decision procedures that can be used to verify a design. As illustrated by the examples in Section 4, both first- and higher-order logic have been successfully employed for hardware verification.

3.3.2 Theorem-Proving Systems. In this section we give brief descriptions of three commonly used automated theorem provers.

Higher Order Logic (HOL). The HOL system [Gordon 1988] is a general-purpose theorem prover whose underlying logic, also called HOL, is a variation of Church's theory of simple types [Church 1940], a higher-order logic. HOL is a descendant of the earlier LCF system [Gordon et al. 1979].

HOL supports both forward and goal-directed backward proofs in a natural-deduction-style calculus. HOL does not use decision procedures (other than complete search strategies); ultimately, all theorems are proven in terms of the axioms and basic inferences of the calculus. Note that, once proven, a theorem can be used in further proofs without recomputation of its own proof. The user interacts with HOL through the functional metalanguage ML. The system is guided by applying *tactics* to proof obligations; a tactic corresponds to a high-level proof step and automatically generates the sequence of elementary inferences necessary to justify the step. Tactics can be composed into even larger steps using *tacticals* such as "apply tactics *A*, *B*, and *C* repeatedly until no further simplification is obtained." An important aspect of the system is that user-defined tactics cannot compromise the soundness of a proof because the basic inferences operate on proof states implemented as an abstract data type in ML. Therefore, valid theorems can be created only through application of the basic inferences.

PVS. The logic of PVS (Prototype Verification System) [Owre et al. 1992; 1996] is a strongly typed higher-order logic with a rich type system, which includes dependent types and predicate subtypes. Type checking in this logic is undecidable and requires the assistance of the theorem prover and possibly human intervention to discharge automatically generated type correctness conditions. Definitions and theorems can be grouped into parameterized theories whose parameters may have constraints attached.

Proofs in PVS are backward proofs using a sequent representation for proof obligations. The inference rules operate at a higher level than the

primitive inferences of higher-order logic and include Skolemization and instantiation, application of theorems, equality rewriting, as well as complex propositional rewriting such as case-split and IF-lifting. Higher-level proof strategies analogous to HOL tactics can be constructed from the basic inference rules using a specialized strategy language.

To discharge certain classes of obligations, PVS employs an integrated decision procedure for equality reasoning, linear arithmetic, arrays, etc., as well as a BDD-based procedure for propositional logic. The decision procedures are integrated with the type checker to make use of additional constraints available from type information. PVS also includes a model checker for finite-state μ -calculus that can be accessed through a formalization of the μ -calculus within the PVS logic [Rajan et al. 1995].

Boyer-Moore. The Boyer-Moore logic is a first-order, quantifier-free logic of total, recursive functions with equality and mathematical induction [Boyer and Moore 1979]. The syntax of the logic is that of applicative Common Lisp. *Nqthm* is a mechanization of this logic [Boyer and Moore 1988]; a re-engineered and slightly extended version was released recently under the name *ACL2* [Kaufmann and Moore 1994; 1996]. The description here is based on *ACL2*.

ACL2 contains axiomatizations of primitive data types such as numbers and lists. New functions can be introduced through a definitional principle that requires proof of termination for recursively defined functions and ensures that consistency is maintained.

ACL2 combines backward and forward methods to prove theorems. In the backward phase, a pending obligation is chosen and the prover attempts to discharge it by applying a sequence of more and more general proof techniques. First, a simplifying stage applies conditional and congruence-based rewrites, a BDD-based propositional decision procedure, a linear arithmetic decision procedure, and other simplification techniques to the obligation. If this step fails to reduce the obligation to *true*, *ACL2* tries other proof techniques, the most general of which attempts to discover an induction scheme for the obligation.

The forward aspect of *ACL2* is that most of the proof techniques are rule-driven. Previously proved theorems can be turned into rules that are added to a rule database and can be used by later proofs. For example, once one has proved the associativity of a certain operator, this theorem can be converted into a rewrite rule that will then be used by the simplifier to rewrite occurrences of this operator in an obligation.

The user guides the theorem prover by starting with simple lemmas that can be proven directly and converted into new rules. These rules are then used to prove successively more difficult lemmas until finally the main theorem is proven. *ACL2* allows definitions, lemmas, and theorems to be collected in *books*. A book may have local lemmas that are invisible outside the book and whose only purpose is to guide the prover to the proof for the main theorems exported by the book. This feature (which was not available in *Nqthm*) facilitates the development of reusable theories.

3.3.3 Proof Strategies for Hardware Verification. Proof discovery in many theorem provers is guided by the user through the application of proof strategies or tactics. Standard tactics operate at a level of detail such as quantifier instantiation or rewrite with respect to a set of equalities. Hardware proofs in a particular application domain often follow common general patterns, which suggests the development of proof strategies that automatically discharge more complex obligations.

Aagaard et al. [1993] report on a hardware proof strategy for both HOL and Nuprl that automatically solves simple goals of the form of Eq. (6); [see p.143]. The tactic automatically expands definitions used in the implementation, then applies heuristics for instantiating quantifiers and for case analysis. Finally, it attempts to discharge obligations through rewriting.

Cyrluk et al. [1994] describe a similar proof strategy implemented in PVS. This strategy applies to correctness conditions similar to (6), where both the specification and implementation are given in terms of a state transition function. A naive strategy would fully expand the definitions of the specification, the implementation and the abstraction mapping, subject the result to case analysis and verify all the cases using the decision procedures. However, the resulting intermediate expression becomes much too large for nontrivial examples. The proof strategy described in Cyrluk et al. [1994] first applies a limited form of rewriting and performs a case-split on the number of cycles the implementation has to run. It then successively applies first a simple and, if this fails, an aggressive rewrite strategy, further case-split and decision procedures to the resulting cases. This strategy can completely and automatically verify the Tamarack microprocessor [Joyce 1988], a relatively simple microcoded design often used as a benchmark example. A similar approach is described in Kumar et al. [1993].

3.4 Combining Model Checking and Deductive Reasoning

In the previous sections, we surveyed formal verification techniques ranging from highly automated, state-exploration-based approaches to theorem-proving-based methods that require a considerable amount of human interaction. A verifier faces the trade-off between choosing an automated technique, which is in general subject to limitations in applicability (such as being confined to finite systems), and more general, but also more work-intensive, approaches.

However, this trade-off can be avoided in certain cases where it is possible to apply automated techniques to subsystems or simpler obligations and then use deductive approaches to combine the results thus obtained into an overall correctness result. In this section, we present tools based on this approach. Related work was reported in Kurshan and Lamport [1993]; Joyce and Seger [1993]; Rajan et al. [1995]; and Schneider and Kropf [1996].

Note that it is a nontrivial exercise to add a model-checking decision procedure to an existing general-purpose theorem prover, since it generally

requires formalizing the semantics of the temporal logic within the logic of the theorem prover (cf., Joyce and Seger [1993]; and Rajan et al. [1995]).

3.4.1 *Compositional Theory for Trajectory Assertions.* Hazelhurst and Seger [1994; 1995] describe a proof system for trajectory assertions consisting of seven inference rules, which in general have the form

$$\text{If } M \models [A_i \Rightarrow C_i] \text{ for } i = 1, \dots, n \text{ and } Q_1, \dots, Q_m, \text{ then } M \models [A \Rightarrow C],$$

i.e., $M \models [A \Rightarrow C]$ can be derived from a set of trajectory assertions that are known to hold, given that the additional properties $Q_1, \dots, Q_m$ are satisfied. Based on assertions verified using STE, more complex properties can be derived using these inference rules. The completeness of this proof system is discussed in Zhu and Seger [1994].

An LCF-style proof tool called VossProver was developed on top of the Voss verification tool. Theorems $M \models [A \Rightarrow C]$, which have already been proven, are represented as objects of an abstract data type in Voss' metalanguage fl. For each inference rule, there is a function (proof rule) that takes the theorems corresponding to the rule's premises as arguments, checks whether the rule is applicable, and if so returns a new theorem corresponding to the rule's conclusions. In addition, there is a basic rule that returns a theorem for a given trajectory assertion after verifying this assertion using STE. Higher-level proof steps based on the above inference rules can be implemented in fl; the implementation of theorems as abstract data types ensures their soundness.

3.4.2 *STeP.* STeP [Manna et al. 1994; Bjørner et al. 1996] is an integrated system for reasoning about reactive systems. The underlying computational model is that of transition systems with fairness constraints; programs are specified in a programming language called SPL, which provides, in addition to sequential constructs such as loops, constructs for concurrency and nondeterminism. Specifications are given in first-order LTL (i.e., quantification over state variables is allowed).

STeP integrates model-checking and theorem-proving methods for proving that a temporal logic formula φ is valid for a program $\mathcal{P}$. The model checker is based on the construction of the product automaton for $\mathcal{P}$ and $\neg \varphi$ and checking the emptiness of its language (see Section 3.2.3). The model checker may be applied to systems with infinite state spaces, in which case termination is not guaranteed.

The theorem proving support in STeP has several aspects. There are proof rules that allow the validity of a temporal formula to be decomposed into so-called *verification conditions* involving only first-order formulas not containing temporal operators. A verification condition has the form of a Hoare triple $\{\varphi\}\tau\{\psi\}$ [Hoare 1969] and asserts that taking a transition τ from a state satisfying φ must lead to a state satisfying ψ . The verification conditions for a program can be visualized using *verification diagrams*,

graphs whose nodes and edges are labeled with formulas and transitions, respectively.

An automatic prover for first order logic, which is integrated with decision procedures for Presburger arithmetic, is available to prove the resulting obligations. If the automatic prover is unable to discharge the obligations, an interactive sequent-style prover may be used to simplify proof obligations.

STeP also provides methods for automatically discovering invariants, which are needed, e.g., for proofs of safety properties. The system attempts to generate invariants both in a bottom-up fashion based on static analysis of the program, as well as top-down by strengthening given properties.

4. CASE STUDIES

In the previous sections, we introduced a variety of specification styles and verification methodologies. We now present selected case studies; we attempted to choose applications to nontrivial, mostly industrial designs in order to provide a sense of the state-of-the-art in verification and how designs have been specified and verified in practice. The application areas include microprocessors, floating-point hardware, protocols, memory arrays, and communications hardware.

4.1 Microprocessors

This section presents recent applications of formal methods to microprocessors. Much of the work has used an approach where the specification is given in terms of a state transition function on the programmer-visible state of the processor. This function captures the effect of executing exactly one instruction. Then, theorem-proving techniques are employed to show that an implementation is a refinement of the specification; the implementation is described at a functional unit, register-transfer, or even gate-level. Earlier work in this category includes the Tamarack [Joyce 1988] and Viper [Cohn 1988; 1989a] verification efforts. Section 4.1.5 describes a verification effort aimed at a lower level, where the specification consists of an RTL description, and state-machine equivalence checking is used to verify an implementation that is partly synthesized and partly hand-crafted.

4.1.1 The Generic Interpreter Theory and AVM-1. Windley [1995a] developed a framework for the verification of refinement between deterministic finite state machines whose transitions can be grouped into classes; this specifically applies to microprocessors, where the classes correspond to instructions. His *generic interpreter theory* (GIT) provides a structured model for the specification of such state machines at different levels of abstraction. The GIT model is based on the common structure of earlier microprocessor verifications such as Tamarack [Joyce 1988]; Viper [Cohn 1988]; and FM8501 [Hunt 1994].

The AVM-1 is a general-purpose 32-bit microprocessor devised for use as an example to demonstrate verification in the GIT framework [Windley 1995a]. AVM-1 was designed to provide a reasonable set of features found

in commercial processors and yet be verifiable at the same time. It has a RISC architecture, a large register file, supervisory mode, and supports external and software interrupts.

The generic interpreter framework permits hierarchical specifications where a design is described at a series of more and more detailed levels of abstraction: for instance, the AVM-1 was specified behaviorally at the top-most level representing the programmer's view of the processor, the microcode level, the multiphase clock level, and with a structural representation at the lowest level [Windley 1995a]. The verification of such a hierarchy proceeds by showing that each of the levels (except for the top-most) implements the next higher level.

In the GIT framework, a deterministic machine is modeled as an *interpreter*. An interpreter $\mathcal{I}[s, e]$ is defined as a predicate over a state stream $e : \mathbb{N} \rightarrow \mathcal{S}_S$ and an environment stream $e : \mathbb{N} \rightarrow \mathcal{S}_E$, as in Section 2.2.2:

$$\mathcal{I}[s, e] = \forall_{t \in \mathbb{N}} s(t+1) = \mathcal{N}(s(t), d(t)), \quad (13)$$

where $\mathcal{N}$ denotes the system's state transition function. For a behavioral description of a microprocessor, it is natural to structure $\mathcal{N}$ into a set $\mathcal{J}$ of transition functions, each of which corresponds to a single machine instruction. In the GIT, $\mathcal{N}$ is expressed in terms of a function that selects the next instruction to be executed in a given state and the instruction-specific transition functions themselves.

The general condition for refinement (i.e., Eq. (6), p. XX) is specialized, using separate functions for temporal and data abstractions:

$$\begin{aligned} \exists_{A_S, A_E, T} \forall_{s_{imp}, e_{imp}} \mathcal{I}_{imp}[s_{imp}, e_{imp}] \\ \Rightarrow \mathcal{I}_{spec}[A_S \circ S_{imp} \circ T, A_E \circ e_{imp} \circ T]. \end{aligned} \quad (14)$$

Here, A_S and A_E map implementation states into specification states and T maps specification time into implementation time. Note that, in fact, $\mathcal{I}_{imp}$ need not be an interpreter of the form (13). In particular, if the implementation is described at a structural level, the predicate $\mathcal{I}_{imp}$ can be expressed as the composition of predicates for the registers, gates and functional units that make up the circuit, with internal wires hidden by existential quantification (see Section 2.2.2).

The specific structure of the state transition function and the abstraction functions allows the correctness statement to be reduced to three lemmas, the most significant of which essentially states that each instruction completes in a specific number of implementation time units and has the effect stipulated by its instruction-specific transition function. This facilitates case analysis over the set of instructions. The definitions of the transition function and the abstraction functions, as well as the above lemmas, are collected into an abstract theory [Windley 1992], which provides a template both for the necessary definitions and the structure of

the proof. Instantiating the abstract theory automatically generates proof obligations for the lemmas; once these obligations are proved, instantiations of the theory's theorems, in particular the main correctness result (14), are established automatically.

4.1.2 Pipelined Microprocessors. One of the major difficulties in the verification of pipelined or super-scalar processors is that instruction boundaries at the specification and the implementation level do not coincide in general. Instead, several partly completed instructions may be executed in parallel in each cycle. Thus, the abstraction function must combine temporal and data aspects [Windley 1995b]. This poses two challenges: it is more difficult to find the appropriate abstraction function and the proof structure becomes more irregular [Windley and Coe 1994].

Burch and Dill [1994] describe an approach to obtaining an abstraction function applicable if the implementation of the processor can be stalled. *Stalling* the processor refers to operating it in a mode where no new instructions are inserted into the pipeline, but the instructions already in the pipeline continue executing as usual. This can be achieved, e.g., by inserting no-op instructions. The pipeline is *flushed* by stalling it sufficiently many cycles to allow all instructions to complete. The abstraction function is then composed of a function that symbolically flushes the pipeline and a pure data-abstraction function.

Their correctness condition states that the application of the specification and the implementation state transition function on an arbitrary state commutes with the above abstraction function, that is, in the notation from the previous section,

$$\forall_{s \in S_{imp}} \exists_m Abs(\mathcal{N}_{imp}(s)) = \mathcal{N}_{spec}^m(Abs(s)). \quad (15)$$

Here, *Abs* is an abstraction function that first flushes the pipeline and then applies the data abstraction mapping from implementation states into specification states. The formulation in (15) includes a recent extension to the model [Burch 1996], which accounts for super-scalar execution, where more than one instruction can be issued per implementation cycle. This is reflected in the variable *m*, which corresponds to the number of instructions issued into the pipeline in an implementation cycle. The correctness statement used in Burch and Dill [1994] is obtained by fixing *m* = 1.

Burch [1996] gives a decomposition theorem that breaks the correctness condition (15) into three simpler properties. Informally, the first property states that the implementation executes instructions correctly from a flushed state; the second confirms that stalling does not affect the specification state; and the third checks that instructions are fetched correctly.

Burch and Dill [1994] and Burch [1996] have applied this methodology to pipelined and super-scalar implementations, respectively, of a subset of the DLX architecture. The DLX [Hennessy and Patterson 1990] is a generic RISC architecture representing common features of (earlier) commercial

RISC architectures such as the MIPS R3000, SPARC1, IBM 801, and Intel i860.

The verification was carried out using a tool based on symbolic execution and a validity checker. Representations of the state transitions functions are compiled from descriptions in a simple HDL. The tool is targeted at the verification of pipeline *control*; uninterpreted function symbols are used to represent, e.g., ALU operations. The proof obligations are expanded and then discharged using a specially tailored decision procedure for quantifier-free, first-order logic with equality and uninterpreted function symbols [Jones et al. 1995; Barrett et al. 1996]. The tool does not allow for deductive reasoning to discharge obligations; however, the user can suggest case splits to the validity checker, which was indeed necessary to complete the verification of the super-scalar DLX implementation.

Other work on pipelined microprocessors includes Srivas and Bickford [1990]; Tahar and Kumar [1993]; Saxe et al. [1993]; Windley [1995b]; and Levitt and Olukotun [1996].

4.1.3 FM9001. The FM9001 is a general-purpose 32-bit microprocessor designed with verification in mind by Brock and Hunt at Computational Logic, Inc. [Brock et al. 1994]. The FM9001 instruction set is based on the instruction set of the earlier FM8502 design [Hunt 1989], and features the usual integer arithmetic and Boolean operations, conditional stores, and five addressing modes. Instructions are of fixed sized and use a two-address format.

The high-level specification for the FM9001 was formalized in Boyer-Moore logic. The central part of the specification is a function FM9001-step which, given the current programmer-visible state of the processor (i.e., registers, flags, and memory), returns the updated state resulting from the execution of one instruction. This function is written in terms of auxiliary functions specifying how the current instruction is fetched from memory, decoded, and executed. The top level-specification is a function that runs FM9001-step for n steps.

Unlike earlier verification efforts, Brock et al. [1994] formalize the implementation-level description not directly as functions in the logic. Instead, they define a netlist HDL whose syntax and semantics were formalized in Boyer-Moore logic [Brock and Hunt 1997]. The formalization consists of a predicate that recognizes well-formed netlists and a simulator called DUAL-EVAL, which defines the operational semantics of the HDL. The DUAL-EVAL netlist description of the processor was translated automatically (but informally) into a commercial netlist format for fabrication.

The implementation and specification are related by a proof carried out in the Nqthm prover. The main result states that for an arbitrary user-visible state s and a positive integer n , there exists an integer c such that executing the specification FM9001-step n steps starting in state s has the same effect on the user-visible state as simulating the behavior of the FM9001 netlist under DUAL-EVAL semantics for c clock cycles, subject to an appropriate abstraction mapping between states at the specification and

the netlist level. In addition, the proof shows that the FM9001 implementation can be reset from an arbitrary, undefined state. This proof is carried out at the netlist level and draws on monotonicity properties proved about the DUAL-EVAL simulator.

The processor was fabricated from its netlist description and was subjected to extensive testing; no functional design errors were reported [Albin et al. 1995].

4.1.4 AAMP5. The AAMP5 microprocessor is a commercial microprocessor distributed by Collins Commercial Avionics and is part of a family of processors intended for use in critical applications such as avionics displays. The architecture was designed as a target architecture for high-level block-structured languages and provides hardware support for runtime-system primitives such as procedure-state saving and parameter passing as well as scheduling primitives such as task-state saving, context switching, and interrupt handling. The instruction set of the AAMP architecture is large and CISC-like with 209 variable-length instructions.

The implementation-level architecture of the AAMP5 consists of four independent units: a bus interface, an instruction cache, a look-ahead fetch unit, and a microcoded, three-stage-pipelined data-processing unit. The implementation consists of ca. 500,000 transistors.

A partial verification of an RTL description of the AAMP5 against a high-level specification was carried out in PVS in a cooperation between Collins, NASA Langley, and SRI International [Miller and Srivas 1995; Srivas and Miller 1996]. The formal specification of the AAMP5 is given in terms of next-state functions that describe the effect of each instruction on a state-tuple representing the programmer-visible state of the processor and the memory. Out of a total of 209 instructions, 108 were specified; the specification consists of ca. 2,500 lines of PVS. A significant part of the specification effort was invested in validating the specification with domain experts to ensure that the formal specification indeed captures the intended behavior as described in the programmer's manual.

At the implementation level, the data-processing unit (DPU) is described at the register-transfer-level, while the other three units are specified only in terms of their external behavior. The processing unit itself is specified as the composition of register-transfer-level components (latches, ROMs, multiplexers, shifters, register files, etc.), and modules such as the ALU described at a more abstract level. There is no formal connection to the gate-level implementation of the processor; this link was validated using traditional, informal design reviews.

The correctness condition used in this verification states that each instruction satisfies the following assertion: if the DPU is in a state where it just begins executing the first microinstruction of an instruction, then execution will eventually complete; the sequence of microinstructions will have the same effect under the abstraction mapping as the macro- (specification-level) instruction; and the DPU will end up in a state where it just begins executing the first microinstruction of the next macroinstruction. In

addition, it has to be shown that the processor can be reset into a valid state. The proofs are complicated by two features of the architecture. First, the pipelined implementation leads to a complex, nonorthogonal abstraction mapping. Second, the decoupling of the bus interface and the instruction prefetch unit has the effect that the number of cycles an instruction takes to complete becomes nonconstant (though finite).

4.1.5 PowerPC Transistor-Level Verification. Appenzeller and Kuehlmann [1995] report on the application of formal methods in the PowerPC microprocessor design process. In this project, the top-level formal specification consists of the RTL description of the design, written in VHDL. The transistor-level implementation was formally verified to be functionally equivalent to this specification using the Verity tool (see Section 3.2.2.1, Kuehlmann et al. [1995]).

The RTL specification was written with simulation performance in mind and validated to conform with the informal PowerPC architecture specification through extensive simulation. The RTL specification is structured into hierarchical layers; the structural decomposition was planned such that the individual modules were manageable in complexity both for the designer and the verification tool. There are three main layers: The top-most layer specifies how the entire processor is composed of its functional units; the second layer describes these units in terms of components, which are in turn specified at the lowest layer.

The transistor-level implementation was developed using the structural decomposition given by the RTL specification. The design consists of a combination of synthesized and custom-designed components and includes both static and precharged circuits. The Verity tool was used to show functional equivalence for all synthesized components and 81% of the custom-designed components. The unverified components either included large storage arrays or were components designed before the introduction of Verity into the design process and would have required restructuring for successful verification. The authors note that much of the verification was possible because Verity was part of the design flow, and circuits were structured with verification in mind. Ninety-five percent of the circuits were verified in less than 800 CPU seconds and 30MB of memory each, which permitted a largely interactive use of the tool in the design phase.

4.2 Floating-Point Units

In the wake of the highly publicized flaw in the Intel Pentium division unit, many researchers looked to using formal methods to verify hardware for floating-point operations. The verification of floating-point *algorithms* also received much attention recently, but is beyond the scope of this paper. Recent efforts in this area include work on SRT division (for the algorithm used in the Pentium floating-point unit, see Cavanagh [1984]) [Miner and Leathrum, Jr. 1996; Ruess et al. 1996; Clarke et al. 1996]; square root [Leeser and O'Leary 1995], and natural logarithm algorithms [Harrison

1995]; and the core of the AMD5_K86 floating-point division algorithm [Brock et al. 1996; Moore et al. 1996].

Bryant [1996] uses BDDs to verify properties of a single iteration of a gate-level implementation of the SRT algorithm.

4.2.1 Intel Extended-Precision FPU. Chen et al. [1996] apply word-level model checking [Clarke et al. 1996] to the verification of all operations of the floating-point unit of a recent Intel microprocessor.⁶ They also demonstrate their approach using the Weitek WTL3170/3171 Sparc floating-point coprocessor⁷ as an example.

Word-level model checking is not capable of directly verifying that the circuit implements a high-level specification of (e.g., IEEE conformant) floating-point operations. Instead, the overall correctness condition is decomposed into a set of properties about subunits of the FPU. These properties may be further decomposed into several assertions, which are then expressed as temporal formulas in word-level CTL. For iterative algorithms, such as division and square-root, these individual properties state that the circuit maintains the appropriate loop invariants. In the case of the Intel FPU, over 120 such properties were verified and are claimed to cover all arithmetic operations of the unit.

In order to make the verification tractable, a technique called *property-specific model extraction* was applied to the circuit description before model checking of the individual verification conditions. This step exploits that a given operation (such as multiplication) usually exercises only part of the FPU; thus, the irrelevant parts of the unit can be disregarded when checking a property pertaining to this operation.

4.2.2 The ADK IEEE Multiplier. Aagaard and Seger [1995] report on the design and verification of an IEEE-compliant floating-point multiplier called ADK. The design satisfies all requirements stated in the IEEE standard, except that some nonstandard rounding modes are not supported and denormalized operands are treated by either considering them as equal to zero (in violation of the standard) or by raising an exception that allows treatment in software.

The ADK is implemented as an array multiplier with radix-8 modified Booth recoding, and has a four-stage pipelined architecture. In the first pipeline stage, the operands are prepared, which involves computing Booth recoding [Cavanagh 1984] of the first operand's significand; in addition, the exponents are added and special cases (such as NaNs) are detected. The three remaining stages consist of the multiplication array of carry-save-adders (stage 2), the carry-propagate adder and normalization unit (stage 3), and the rounding unit (stage 4). The design was implemented in structural VHDL and synthesized into a unit-delay gate-level model with ca. 33,000 gates.

⁶ The paper does not specify exactly which model.

⁷The architecture of this unit has been documented in the literature [Birman et al. 1990].

The top-level specification of the multiplier represents a formalization of the IEEE multiplication standard. It is given as a relation between input operands and results and is written in terms of arithmetic operations on integers, i.e., the significand and exponent vectors are interpreted as integer values at the specification level.

The verification was carried out in the VossProver system (see Section 3.4.1). First, symbolic trajectory evaluation was used to verify that the gate-level implementations of the design's functional units satisfy corresponding TF specifications. Stages three and four were handled together by a single trajectory evaluation. Separate trajectory theorems were verified for the Booth recoder, the multiplier array, and the remaining functional units. The trajectory theorems thus obtained for the individual units were composed into specifications of larger circuit components and eventually shown to imply the top-level specification. Relating the trajectory theorems (stated in terms of bit-vectors) to the specification (given in terms of integers) required several lemmas on properties of bit-vectors, which were discharged using a specialized decision procedure.

4.3 Asynchronous and Distributed Systems

The main difficulty in designing systems consisting of independent, interacting components lies in the complexity arising from the often extremely large number of possible orderings in which actions of the individual components can interleave. Subtle bugs can appear in an incorrect design when events take place in an order unforeseen by the designer. Due to the large number of possible event orderings, informal validation methods such as simulation are often incapable of exposing design errors. In this section, we present applications where the correct interoperation of system components was specified in temporal logic and verified by model checking.

4.3.1 The Summit Bus Converter. Harkness and Wolf [1994] apply symbolic model checking to the Summit bus converter, an industrial component of a multiprocessor system. The bus converter provides a communication link between a high-speed processor bus and a low-speed I/O bus. Both the processor bus and the I/O bus components of the converter are designed as collections of interacting finite-state machines. The authors informally abstracted away aspects of the design that were considered unimportant to the verification conditions they had in mind. For instance, error handling, data transfer, pipelined link transfer, and other performance-enhancing features were not modeled, and addresses were reduced to one bit. Model checking was then applied to the resulting abstract model of the design, which was formalized in SMV to demonstrate the presence of deadlock on the system's link queues in earlier versions of the design, as well as the absence of deadlock in a revised version. This was accomplished by showing that the system satisfies the CTL formula

$$\mathbf{AG}(LQ.Head.Occ \Rightarrow \mathbf{AF}(\neg LQ.Head.Occ)),$$

stating that whenever the head buffer of the link queue is occupied with a request, this will eventually be processed.

The potential for deadlock in the incorrect design was already known, and the abstract model was created with the verification of deadlock-freedom in mind. When validating their model against other putative temporal properties of the system, the authors also discovered a data corruption error in the design.

4.3.2 Cache Coherence Protocols. A distributed shared memory system (see, e.g., Lenoski et al. [1992]) consists of a collection of clusters, each of which has one or more processors, local memory, caches, memory-mapped I/O devices, a directory, and a DSM controller. The clusters are connected through a communication fabric. The DSM controller ensures that each processor sees the same, shared, address space. The cache coherence protocol is required to ensure that this memory is *coherent*: although memory reads and writes may occur concurrently on different nodes, the values obtained from reads appear as if the memory actions occurred in a single, global, sequential order.

Model-checking techniques were used to verify several such protocols. Cache coherence protocols are typically specified as a collection of communicating state machines, making them natural candidates for model checking. However, real implementations have extremely large amounts of state, including several status bits for each cache block on each node. Model checking is generally applied to an informally constructed abstraction of a small system. In this section, we present two recent applications of this approach, the coherence protocols of a recent Silicon Graphics multiprocessor architecture and of the Sun S3.mp architecture. Numerous other coherence protocols have been verified, including those for the Gigamax [McMillan and Schwalbe 1991], Futurebus+ [Clarke et al. 1993], and SCI architectures [Stern and Dill 1995].

Eiríksson and McMillan [1995] describe the verification of the cache coherence protocol for a Silicon Graphics multiprocessor using the SMV model checker. The state transition tables for the protocol are machine translated into SMV representations and composed with a model of the connection network. Then, high-level correctness properties of the protocol, expressed in CTL, are verified. To make the verification tractable, the high-level specification was kept as abstract as possible and dependent variables were eliminated (see Section 3.1.2.3). Verification was possible for models consisting of up to four clusters. The properties verified included absence of deadlock, requests always receive the correct response, absence of unsolicited responses, and safety properties expressing cache coherence. In one case, a bug that lead to loss of coherence was exposed; the problem occurred after a sequence of 19 events in a particular ordering, and would most likely not have been found in simulation [Eiríksson and McMillan 1995].

The verification effort was an integral part of the design process [Eiríksson 1996; Eiríksson and McMillan 1995]. A single abstract specification of

the cache coherence protocol was used for formal verification, performance evaluation, documentation, and design. For a number of subcomponents, it has also been verified that a Verilog RTL implementation of the component is a refinement of the corresponding high-level FSM. This was accomplished using a translator that extracts an SMV model from the Verilog code and verifying the refinement property with model checking. The correctness of the evolving design was tracked by periodically rechecking the complete set of verification conditions. A complete regression run that checks all of the ca. 300 CTL assertions took less than two days on a cluster of 200 workstations.

The Sun S3.mp shared-memory multiprocessor [Nowatzky et al. 1995] also uses a directory-based cache-coherency scheme. The nodes in this system are standard workstations with special interconnect controllers attached to their system buses. Each workstation may itself have more than one processor; consistency within a single workstation is achieved through a snooping protocol on the system bus. Each memory block has a home node, and the DSM controller on that node maintains a list of other nodes that contain copies of the block. The protocol allows one processor to have an exclusive copy of a block or many processors to have read-only copies. The home node is responsible for serializing requests for its memory blocks.

Pong et al. [1995] model the protocol in the Mur ϕ language and system [Dill et al. 1992] (see Section 3.1.5). Several simplifying assumptions were made in order to make the verification tractable. Only one memory block is modeled; it is assumed that other blocks do not interfere. The model contains this single block's home node with its directory entry as well as k other nodes with one cache block each. The nodes have only one processor, based on the assumption that the local snooping protocol works correctly. The network is represented by non-order-preserving buffers for incoming and outgoing messages at each node. The data values held in the cache are represented by five symbolic values reflecting their relation with the other cached copies. For example, the symbolic value `GlobalFresh_HOLD` denotes a value written to the cache by the local node but which has not yet been propagated to all other nodes; similarly, `Obsolete` denotes a value that is out of date. A required safety property for a protocol that maintains data consistency is that processors can never read an `Obsolete` value. The symmetry of the model is exploited to group equivalent symmetric states [Ip and Dill 1996a]. Using this model, several protocol errors were detected and corrected. Data consistency was verified for a model with two remote nodes in addition to the home node.

To verify larger configurations, the authors applied a technique called *symbolic state model* (SSM) [Pong and Dubois 1993], which is based on the observation that it is irrelevant precisely how many nodes hold a shared copy of a block; rather, it is sufficient to distinguish states in which either zero, one, or more nodes have caches and network buffers in a particular state. In the SSM for the S3.mp cache coherence, a system state consists of

a detailed account of the nodes on the distributed linked list, as well as repetition counts in the above sense for the remaining nodes. Using SSM, the correctness condition is established for a configuration with arbitrarily many nodes, but where at most four nodes (including the home node) can simultaneously share a copy of the block. This model led to the discovery of protocol errors that were not detected with the non-SSM model described above.

4.4 Memory Subsystems

Complex memory arrays such as multiported register files and caches are particularly difficult applications for model-checking techniques because of their very large state spaces. Furthermore, their implementations typically have nontrivial timing characteristics that may preclude the extraction of an accurate gate-level model from the transistor netlist.

Pandey et al. [1996] apply symbolic trajectory evaluation to two arrays from the PowerPC architecture. The first array is a multiported register file with 36 32-bit registers and two write and five read ports. The second unit is a data cache tag (DTAG) with 128 4-way associative sets. Each way in a set has a 20-bit tag plus valid and dirty bits; in addition, each set has 6 LRU bits. When a 7-bit index to select one of the sets as well as a tag are presented to the unit, it determines whether the tag matches one of four ways. If so, this is indicated and the LRU history of the set is updated. Otherwise, the tag of the least-recently used way is output. The implementation of the DTAG has over 12,000 latches.

The specifications for the units were structured into high-level assertions about abstract state transitions (e.g., memory read or write) and an implementation mapping that captures the detailed timing information [Beatty and Bryant 1994]. The high-level assertions have the form $A \xRightarrow{\text{leads to}} C$, where the antecedent A is an assertion about the inputs and internal state before an abstract transition and C is a corresponding assertion about the internal state afterwards. The implementation mapping relates each clause in A or C to a trajectory formula that specifies the detailed timing of the node(s) corresponding to the high-level clause. Assertions on the *pre* and *post* values of the registers are expressed in the form $R[i] = u$, where i is a symbolic index [Beatty 1993], and the corresponding circuit nodes are initialized with symbolic expressions rather than constants. This technique is vital to the tractability of the verification in that it reduces the number of symbolic variables to a number approximately logarithmic in the number of registers.

The implementations were given both as flat transistor netlists and at the RTL level in an internal HDL. To identify the state-holding nodes in the transistor netlist, an automated technique was used that exercises a write operation on the design with symbolic input values and then matches the resulting symbolic expressions on the nodes against symbolic indexing functions for the state-holding elements. The switch-level models for the symbolic simulation required for trajectory evaluation were derived from

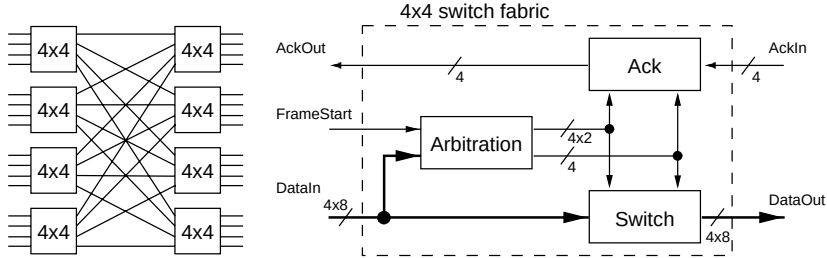


Fig. 7. Delta network and 4x4 switch fabric.

the netlists. In the case of the DTAG unit, some amount of manual modeling was necessary, which involved annotating the circuit with transistor strengths. Care was taken not to assign directions to transistors.

Both the switch-level and the RTL-level models were checked against the trajectory assertions comprising the circuit specification. For both units, discrepancies between the behavior of both models appeared, although in the case of the register file, these were only observed under environment behaviors not allowed by the specification. For the DTAG unit, two actual bugs were found, the first of which was due to a transistor “sneak path,” and therefore was not visible at the RTL level. Checking the most complicated assertion for the DTAG unit required ca. 10 minutes of processing time and 150MB of memory.

4.5 ATM Switch Fabrics

The asynchronous transfer mode (ATM) is a technology for high-speed local- and wide-area networks [Vetter 1995]. Its main characteristic is that data is transported in small, fixed-size packets called *cells*. The network consists of multiported switches interconnected by point-to-point high-speed links. Fairisle [Leslie and McAuley 1991] is an ATM architecture for local area networks (LANs) supporting link speeds of 100Mbps.

Figure 7 shows the architecture of the switch fabric that forms the core of the Fairisle ATM switch and performs the actual routing of cells from one of the sixteen input lines to the desired output lines. The network is implemented as a self-routing delta network of eight 4x4 switch fabrics: port controllers at the input lines prepend two routing tags to incoming cells. The 4x4 fabrics read the tags to determine which of their outputs a cell is to be forwarded to, and then strip off the tag. If two cells have the same destination, one of them must be dropped; successful delivery is communicated back to the input via the acknowledgment lines. The decision as to which of two or more competing cells will be forwarded is made by the arbitration unit, according to a round-robin scheme that also takes priority information from the routing tags into consideration. The frame start signal indicates that new cells will appear at the inputs; cells appear in synch on the input lines. In the following, we discuss two applications of formal methods to the switch fabric.

Tahar et al. [1996] verified safety properties of the 4x4 fabric. They created two models of the design: a gate-level model derived from an HDL description and an abstract model where data inputs and outputs are abstracted to an abstract sort `wordn`, representing n -bit words, and data-path operations are represented by uninterpreted function symbols.

The sequential equivalence of the two models (for fixed $n = 8$) was verified by constructing their product machine. Implicit state exploration of the product machine was performed using multiway decision graphs (MDGs), a generalization of BDDs which supports abstract sorts and uninterpreted functions [Anon et al. 1996; Corella et al. 1997] as the representation for sets of states. To account for the difference in the encoding of data values in the two models, encoding/decoding functions converting between the abstract sort `wordn` and 8-bit vectors were incorporated into the product machine.

The authors then verified a number of properties of the abstract model. Correct operation is required only in an environment that meets certain constraints; for example, the routing tags must appear on the input lines a certain number of clock cycles after the frame start signal was asserted. The environment was modeled by a state machine that generates allowed sequences of input signals. The properties verified were phrased as safety properties of the composition of the environment machine and the abstract model. These safety properties were then verified with implicit state space exploration, again using MDGs as the underlying representation for sets of states. For example, it was verified that proper default values appear at the data and acknowledgment outputs after a reset. Properties regarding the correct forwarding of data require that, after an appropriate delay relative to the beginning of a frame, the appropriate data output equals the corresponding data input from four cycles earlier, subject to arbitration decisions. In order to phrase this condition as a safety property, it was necessary to route copies of the data signals generated by the environment through a four-cycle delay buffer.

Curzon and Leslie [1996] report on the verification of the complete 16x16 fabric in HOL. This verification effort is based on earlier work by the first author on the 4x4 fabric [Curzon 1994]. A hierarchical gate-level description of the implementation is given in the specification style introduced in Section 2.2.2. A behavioral specification of the switching element is given as a predicate relating the input and output signals over time. The overall correctness statement has the form of Eq. (6); the implementation predicate includes clauses that impose constraints on the environment behavior, such as timing properties of the input signals.

The 4x4 elements of the 16x16 fabric are in fact not identical; there are small differences in timing between the two stages, and a slight difference in the routing behavior between the top and bottom elements of the first stage. The proofs for these elements were adapted from the original proof of the 4x4 element. The proof for the main correctness theorem for the 16x16 fabric could not be completed because the theorems proven for the first

stage elements could not guarantee an environment assumption concerning the timing of cell arrivals needed by the second stage.

Based on this proof effort, several modifications of the original 16x16 design were proposed, resulting in simplifications of the interfaces between functional units and the removal of environment assumptions. In particular, the environment assumption that prevented the completion of the original proof could be removed by adding a small amount of logic to the arbitration unit. All but one of the suggested modifications were considered acceptable by the designer of the original 16x16 fabric. The authors conclude that keeping provability in mind during the design phase (as opposed to verification after design completion) can result in shorter proofs, and even improved, simpler designs.

Tahar and Curzon [1996] provide a more detailed comparison of the two verification efforts.

5. CONCLUSION

Traditional simulation-based validation can only check a very small fraction of the behaviors of any nontrivial digital design. Formal verification research aims to verify that designs work correctly for all allowed inputs and all reachable states by using rigorous mathematical methods. This requires a formal, mathematical specification of the design requirements, an accurate model of the implementation, and practical techniques to show that the implementation indeed satisfies the requirements. When a claim is made that a design was formally verified, it is important to understand what requirements were specified and at what level of detail the design was modeled. This paper describes formal methods for specification and verification and summarizes case studies where these methods are applied to industrial-scale designs. There are important areas of formal methods in hardware design that are beyond the scope of this paper, including verification of real-time systems [Alur and Henzinger 1992], advanced techniques for combinational equivalence [Jain et al. 1997], and formal synthesis [Kumar et al. 1996] (the references provide introductions and surveys for the respective areas).

Broadly summarizing, specification techniques tend to describe either temporal properties that a design should have or a high-level model of behavior that the design should implement. For example, a temporal formula could state that all messages that enter a network must eventually be delivered, or that executing an instruction leaves the right symbolic values in the register file. Using high-level models, one could specify the network as a collection of abstract queues with a process that transfers messages between queues according to their routing tags, and the processor could be specified with a high-level model of its instruction set.

Design models can range from very abstract models where the design is divided into a few blocks, down to very detailed descriptions that include every transistor. Verification at a high level of abstraction does not prove that the lower-level details of the implementation are correct. On the other

hand, formal methods can be very effective for finding errors at high levels of abstraction before a large design effort is invested in implementing a flawed system architecture. The choice of models must reflect both what the designer believes is most likely to reveal errors and what is possible to verify with existing methods.

Verification techniques can generally be divided into two categories: reachability analysis (e.g., explicit or symbolic state enumeration) and deductive methods. Model checkers, language containment checkers, and state machine equivalence checkers are examples of the first approach. Model-checking tools are often used to verify requirements specified as temporal properties, but, as noted in Section 3.1.3, model checking can also be used to verify refinement of a high-level model. For these methods to be applicable, the model must be small enough for the verification to complete with the CPU and memory resources available. This usually requires that a design is verified at a relatively abstract level, or that relatively small subsystems of the design are verified separately. Although these methods offer a high degree of automation, successful verification often requires an expert's insight into the design itself as well as the implementation of the model checker.

Many different theorem provers have been used for deductive verification. Most examples using theorem provers verify that some model of a design is a refinement of a higher level, although deductive proofs of temporal properties are also possible [Staunstrup et al. 1992; Bjørner et al. 1996]. These methods are guided by an expert's understanding of the design and are not limited to finite structures, which allows deductive verification to address designs that cannot be verified by more automatic techniques. However, using a theorem prover requires a high degree of expertise in mathematical logic, a familiarity with the theorem prover, and an intimate understanding of the design itself.

In many of the larger verification efforts described in this paper, both model-checking and theorem-proving-based, the developers of the tool (or the underlying theory) were involved in the verification project [Eiríksson and McMillan 1995; Miller and Srivas 1995; Pandey et al. 1996; Brock et al. 1996]. Although these efforts required the expertise of the tool developer, the successful results on real-world designs indicate that formal verification is likely to become more widely adopted in industry at large in the near future.

ACKNOWLEDGMENTS

We thank Alan Hu and Tarik Ono-Tesfaye for many helpful discussions. We also thank the anonymous referees for their constructive comments.

REFERENCES

AAGAARD, M. D. AND SEGER, C.-J. H. 1995. The formal verification of a pipelined double-precision IEEE floating-point multiplier. In *Proceedings of the 1995 IEEE/ACM Interna-*

- tional Conference on Computer-Aided Design (ICCAD-95, San Jose, CA, Nov. 5–9)*, R. Rudell, Ed. IEEE Computer Society Press, Los Alamitos, CA, 7–10.
- AGAARD, M. D., LEESER, M. E., AND WINDLEY, P. J. 1993. Toward a super duper hardware tactic. In *HUG '93. Lecture Notes in Computer Science*, vol. 780. Springer-Verlag, New York, 400–412.
- ABADI, M. AND LAMPORT, L. 1991. The existence of refinement mappings. *Theor. Comput. Sci.* 82, 2 (May 31, 1991), 253–284.
- ABADI, M. AND LAMPORT, L. 1993. Composing specifications. *ACM Trans. Program. Lang. Syst.* 15, 1 (Jan. 1993), 73–132.
- ALBIN, K. L., BROCK, B. C., HUNT, W. A., AND SMITH, L. M. 1995. Testing the FM9001 microprocessor. Tech. Rep. 90. Computational Logic, Inc., Austin, TX.
- ALUR, R. AND HENZINGER, T. A. 1992. Logics and models of real time: A survey. In *Real-Time: Theory in Practice*. Lecture Notes in Computer Science, vol. 600. Springer-Verlag, New York, 74–106.
- ANON, K. D., BOULERICE, N., CERNY, E., CORELLA, F., LANGEVIN, M., SONG, X., TAHAR, S., XU, Y., AND ZHOU, Z. 1996. MDG tools for the verification of RTL designs. In *Proceedings of the 8th International Conference on Computer-Aided Verification (CAV '96, New Brunswick, NJ, July/Aug.)*, R. Alur and T. A. Henzinger, Eds. Lecture Notes in Computer Science, vol. 1102. Springer-Verlag, New York, NY, 433–436.
- APPENZELLER, D. P. AND KUEHLMANN, A. 1995. Formal verification of the PowerPC™ microprocessor. In *Proceedings of the International Conference on Computer Design (ICCD '95, Oct.)*. 79–84.
- BARRETT, C., DILL, D., AND LEVITT, J. 1996. Validity checking for combinations of theories with equality. In *Proceedings of the First International Conference on Formal Methods in Computer-Aided Design (FMCAD '96, Palo Alto, CA, Nov.)*, M. Srivas and A. Camilleri, Eds. Lecture Notes in Computer Science, vol. 1166. Springer-Verlag, New York, NY, 187–201.
- BEATTY, D. L. 1993. A methodology for formal hardware verification, with application to microprocessors. Ph.D. Dissertation. Carnegie Mellon University, Pittsburgh, PA.
- BEATTY, D. L. AND BRYANT, R. E. 1994. Formally verifying a microprocessor using a simulation methodology. In *Proceedings of the 31st Annual Conference on Design Automation (DAC'94, San Diego, CA, June 6–10, 1994)*, M. Lorenzetti, Ed. ACM Press, New York, NY, 596–602.
- BEER, I., BEN-DAVID, S., EISNER, C., AND LANDVER, A. 1996. RuleBase: An industry-oriented formal verification tool. In *Proceedings of the 33rd Conference on Design Automation (DAC '96, Las Vegas, NV, June)*. 655–660.
- BERNHOLTZ, O., VARDI, M. Y., AND WOLPER, P. 1994. An automata-theoretic approach to branching-time model checking. In *Proceedings of the Sixth International Conference on Computer-Aided Verification (CAV'94, June)*. Lecture Notes in Computer Science, vol. 818. Springer-Verlag, New York, 142–155.
- BIRMAN, M., SAUMELS, A., CHU, G., CHUK, T., HU, L., MCLEOD, J., AND BARNES, J. 1990. Developing the WTL3170/3171 Sparc floating-point coprocessors. *IEEE Micro* 10, 1 (Feb.), 55–64.
- BJØRNER, N., BROWNE, A., CHANG, E., COLÓN, M., KAPUR, A., MANNA, Z., SIPMA, H., AND URIBE, T. 1996. STeP: Deductive-algorithmic verification of reactive and real-time systems. In *Proceedings of the 8th International Conference on Computer-Aided Verification (CAV '96, New Brunswick, NJ, July/Aug.)*, R. Alur and T. A. Henzinger, Eds. Lecture Notes in Computer Science, vol. 1102. Springer-Verlag, New York, NY, 415–418.
- BORMANN, J.-R., LOHSE, J.-R., PAYER, M., AND VENZL, G. 1995. Model checking in industrial hardware design. In *Proceedings of the 32nd ACM/IEEE Conference on Design Automation (DAC '95, San Francisco, CA, June 12–16, 1995)*, B. T. Preas, Ed. ACM Press, New York, NY, 298–303.
- BOYER, R. S. AND MOORE, J. S. 1979. *A Computational Logic*. Academic Press, Inc., New York, NY.
- BOYER, R. S. AND MOORE, J. S. 1988. *A Computational Logic Handbook*. Academic Press Perspectives in Computing. Academic Press Prof., Inc., San Diego, CA.

- BRACE, K. S., RUDELL, R. L., AND BRYANT, R. E. 1990. Efficient implementation of a BDD package. In *Proceedings of the ACM/IEEE Conference on Design Automation* (DAC '90, Orlando, FL, June 24-28), R. C. Smith, Ed. ACM Press, New York, NY, 40-45.
- BRADFIELD, J. C. 1992. *Verifying Temporal Properties of Systems*. Birkhäuser User Progress in Theoretical Computer Science Series. Birkhäuser Boston Inc., Cambridge, MA.
- BRAYTON, R. K., HACHTEL, G. D., SANGIOVANNI-VINCENTELLI, A., SOMENZI, F., AZIZ, A., CHENG, S.-T., EDWARDS, S. A., KHATRI, S. P., KUKIMOTO, Y., PARDO, A., QADEER, S., RANJAN, R. K., SARWARY, S., SHIPLE, T. R., SWAMY, G., AND VILLA, T. 1996. VIS. In *Proceedings of the First International Conference on Formal Methods in Computer-Aided Design* (FMCAD '96, Palo Alto, CA, Nov.), M. Srivas and A. Camilleri, Eds. Lecture Notes in Computer Science, vol. 1166. Springer-Verlag, New York, NY, 248-256.
- BROCK, B. AND HUNT, W. A. 1990. Report on the formal specification and partial verification of the VIPER microprocessor. Tech. Rep. 46. Computational Logic, Inc., Austin, TX.
- BROCK, B., HUNT, W. A., AND KAUFMANN, M. 1994. The FM9001 microprocessor proof. Tech. Rep. 86. Computational Logic, Inc., Austin, TX.
- BROCK, B., KAUFMANN, M., AND MOORE, J. S. 1996. ACL2 theorems about commercial microprocessors. In *Proceedings of the First International Conference on Formal Methods in Computer-Aided Design* (FMCAD '96, Palo Alto, CA, Nov.), M. Srivas and A. Camilleri, Eds. Lecture Notes in Computer Science, vol. 1166. Springer-Verlag, New York, NY, 275-293.
- BROCK, B. C. AND HUNT, W. A. 1997. The DUAL-EVAL hardware description language and its use in the formal specification and verification of the FM9001 microprocessor. *Formal Methods Syst. Des.* 11, 1 (July), 71-104.
- BRYANT, R. E., BEATTY, D., BRACE, D. K., CHO, K., AND SHEFFLER, T. 1987. COSMOS: A compiled simulator for MOS circuits. In *Proceedings of the 24th ACM/IEEE Conference on Design Automation* (DAC '87, Miami Beach, FL, June 28-July 1, 1987), A. O'Neill and D. Thomas, Eds. ACM Press, New York, NY, 9-16.
- BRYANT, R. E. AND CHEN, Y.-A. 1995. Verification of arithmetic circuits with binary moment diagrams. In *Proceedings of the 32nd ACM/IEEE Conference on Design Automation* (DAC '95, San Francisco, CA, June 12-16, 1995), B. T. Preas, Ed. ACM Press, New York, NY, 535-541.
- BRYANT, R. E. 1986. Graph-based algorithms for Boolean function manipulation. *IEEE Trans. Comput.* C-35, 8 (Aug. 1986), 677-691.
- BRYANT, R. 1991. A methodology for hardware verification based on logic simulation. *J. ACM* 38, 2 (Apr. 1991), 299-328.
- BRYANT, R. E. 1991. On the complexity of VLSI implementations and graph representations of boolean functions with application to integer multiplication. *IEEE Trans. Comput.* 40, 2 (Feb. 1991), 205-213.
- BRYANT, R. E. 1992. Symbolic Boolean manipulation with ordered binary-decision diagrams. *ACM Comput. Surv.* 24, 3 (Sept. 1992), 293-318.
- BRYANT, R. E. 1995. Binary decision diagrams and beyond: Enabling technologies for formal verification. In *Proceedings of the 1995 IEEE/ACM International Conference on Computer-Aided Design* (ICCAD-95, San Jose, CA, Nov. 5-9), R. Rudell, Ed. IEEE Computer Society Press, Los Alamitos, CA, 236-243.
- BRYANT, R. E. 1996. Bit-level analysis of an srt divider circuit. In *Proceedings of the 33rd Conference on Design Automation* (DAC '96, Las Vegas, NV, June). 661-665.
- BURCH, J., CLARKE, E., LONG, D., McMILLAN, K., AND DILL, D. 1994. Symbolic model checking for sequential circuit verification. *IEEE Trans. Comput.-Aided Des. Integr. Circuits* 13, 4 (Apr.), 401-424.
- BURCH, J. R. 1996. Techniques for verifying superscalar microprocessors. In *Proceedings of the 33rd Conference on Design Automation* (DAC '96, Las Vegas, NV, June). 552-557.
- BURCH, J. R., CLARKE, E. M., McMILLAN, K. L., DILL, D. L., AND HWANG, L. J. 1990. Symbolic model checking: 10^{20} states and beyond. In *Proceedings of the Fifth Annual Symposium on Logic in Computer Science* (LICS '90, June). 428-439.
- BURCH, J. R. AND DILL, D. L. 1994. Automatic verification of pipelined microprocessor control. In *Proceedings of the Sixth International Conference on Computer-Aided Verifica-*

- tion (CAV'94, June). Lecture Notes in Computer Science, vol. 818. Springer-Verlag, New York, 68–80.
- CAVANAGH, J. J. 1984. *Digital Computer Arithmetic: Design and Implementation*. McGraw-Hill, Inc., New York, NY.
- CHANDY, K. M. AND MISRA, J. 1988. *Parallel Program Design: A Foundation*. Addison-Wesley Longman Publ. Co., Inc., Reading, MA.
- CHEN, Y.-A., CLARKE, E., HO, P.-H., HOSKOTE, Y., KAM, T., KHAIRA, M., O'LEARY, J., AND ZHAO, X. 1996. Verification of all circuits in a floating-point unit using word-level model checking. In *Proceedings of the First International Conference on Formal Methods in Computer-Aided Design (FMCAD '96, Palo Alto, CA, Nov.)*, M. Srivas and A. Camilleri, Eds. Lecture Notes in Computer Science, vol. 1166. Springer-Verlag, New York, NY, 19–33.
- CHURCH, A. 1940. A formulation of the simple theory of types. *J. Symb. Logic* 5, 56–115.
- CLARKE, E. M., EMERSON, E. A., AND SISTLA, A. P. 1986. Automatic verification of finite-state concurrent systems using temporal logic specifications. *ACM Trans. Program. Lang. Syst.* 8, 2 (Apr. 1986), 244–263.
- CLARKE, E. M., FUJITA, M., AND ZHAO, X. 1995. Hybrid decision diagrams. In *Proceedings of the 1995 IEEE/ACM International Conference on Computer-Aided Design (ICCAD-95, San Jose, CA, Nov. 5–9)*, R. Rudell, Ed. IEEE Computer Society Press, Los Alamitos, CA, 159–163.
- CLARKE, E., GRUMBERG, O., HIRAISHI, H., JHA, S., LONG, D., McMILLAN, K., AND NESS, L. 1993. Verification of the Futurebus+ cache coherence protocol. In *Proceedings of the 11th Conference on Computer Hardware Description Languages and Their Applications (CHDL '93)*. IFIP, Laxenburg, Austria.
- CLARKE, E., McMILLAN, K., CAMPOS, S., AND HARTONAS-GARMHAUSEN, V. 1994. Symbolic model checking. In *Proceedings of the Sixth International Conference on Computer-Aided Verification (CAV'94, June)*. Lecture Notes in Computer Science, vol. 818. Springer-Verlag, New York, 419–422.
- CLARKE, E. M., McMILLAN, K. L., ZHAO, X., FUJITA, M., AND YANG, J. 1993. Spectral transforms for large boolean functions with applications to technology mapping. In *Proceedings of the 30th International Conference on Design Automation (DAC'93, Dallas, TX, June 14–18)*, A. E. Dunlop, Ed. ACM Press, New York, NY, 54–60.
- CLARKE, E. M., DRAGHICESCU, I. A., AND KURSHAN, R. P. 1990. A unified approach for showing language containment and equivalence between various types of ω -automata. In *Proceedings of the 15th Colloquium on Trees in Algebra and Programming (CAAP'90, May)*. Lecture Notes in Computer Science. Springer-Verlag, New York, 103–116.
- CLARKE, E. M., GERMAN, S. M., AND ZHAO, X. 1996. Verifying the SRT division algorithm using theorem proving techniques. In *Proceedings of the 8th International Conference on Computer-Aided Verification (CAV '96, New Brunswick, NJ, July/Aug.)*, R. Alur and T. A. Henzinger, Eds. Lecture Notes in Computer Science, vol. 1102. Springer-Verlag, New York, NY, 111–122.
- CLARKE, E. M., GRUMBERG, O., AND HAMAGUCHI, K. 1994. Another look at LTL model checking. In *Proceedings of the Sixth International Conference on Computer-Aided Verification (CAV'94, June)*. Lecture Notes in Computer Science, vol. 818. Springer-Verlag, New York.
- CLARKE, E. M., GRUMBERG, O., AND LONG, D. E. 1994. Model checking and abstraction. *ACM Trans. Program. Lang. Syst.* 16, 5 (Sept. 1994), 1512–1542.
- CLARKE, E. M., KHAIRA, M., AND ZHAO, X. 1996. Word level model checking: Avoiding the Pentium FDIV error. In *Proceedings of the 33rd Conference on Design Automation (DAC '96, Las Vegas, NV, June)*. 645–648.
- COHN, A. 1988. A proof of correctness of the Viper microprocessor: The first level. In *VLSI Specification, Verification and Synthesis*, G. Birtwistle and P. Subrahmanyam, Eds. Kluwer Academic Publishers, Hingham, MA, 27–71.
- COHN, A. 1989. Correctness properties of the Viper block model: The second level. In *Current Trends in Hardware Verification and Automated Theorem Proving*, G. Birtwistle and P. A. Subrahmanyam, Eds. Springer-Verlag, New York, NY, 1–91.

- COHN, A. 1989. The notion of proof in hardware verification. *J. Autom. Reasoning* 5, 2 (June 1989), 127–139.
- CORELLA, F., ZHOU, Z., SONG, X., LANGEVIN, M., AND CERNY, E. 1997. Multiway decision graphs for automated hardware verification. *Formal Methods Syst. Des.* 10, 1 (Feb.), 7–46.
- COUDERT, O., BERTHET, C., AND MADRE, J. C. 1989. Verification of synchronous sequential machines based on symbolic execution. In *Automatic Verification Methods for Finite State Systems*. Lecture Notes in Computer Science, vol. 407. Springer-Verlag, New York, 365–373.
- COUDERT, O. AND MADRE, J. C. 1990. A unified framework for the formal verification of sequential circuits. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD'90)*. 126–129.
- CURZON, P. 1994. The formal verification of the Fairisle ATM switching element: An overview. Tech. Rep. 328,. Univ. of Cambridge Computer Laboratory, Cambridge, UK.
- CURZON, P. AND LESLIE, I. 1996. Improving hardware design whilst simplyfying their proof. In *Proceedings of the Third Workshop on Designing Correct Circuits (DCC, Bastad, Sweden, Sept.)*.
- CYRLUK, D., LINCOLN, P., AND SHANKAR, N. 1996. On Shostak's decision procedure for combinations of theories. In *Proceedings of the 15th International Conference on Automated Deduction (CADE'96, July)*. Lecture Notes in Computer Science, vol. 1104. Springer-Verlag, New York, 463–477.
- CYRLUK, D., RAJAN, S., SHANKAR, N., AND SRIVAS, M. K. 1994. Effective theorem proving for hardware verification. In *Proceedings of the Second International Conference on Theorem Provers in Circuit Design (TPCD'94)*. Lecture Notes in Computer Science, vol. 901. Springer-Verlag, New York, 203–222.
- DAC EXHIBITS, 1997. Thirty-Fourth DAC exhibit information. <http://www.dac.com/34exhibits.html>.
- DÉHARBE, D. AND BORRIONE, D. 1995. Semantics of a verification-oriented subset of VHDL. In *Correct Hardware Design and Verification Methods (CHARME '95, Oct.)*. Lecture Notes in Computer Science, vol. 987. Springer-Verlag, New York, 293–310.
- DILL, D. L. 1996. The Mur ϕ verification system. In *Proceedings of the Eighth International Conference on Computer-Aided Verification (CAV '96, Aug.)*. Lecture Notes in Computer Science, vol. 1102. Springer-Verlag, New York, 390–393.
- DILL, D. L., DREXLER, A. J., HU, A. J., AND YANG, C. H. 1992. Protocol verification as a hardware design aid. In *Proceedings of the International Conference on Computer Aided Design*. 522–525.
- DUFFY, D. A. 1991. *Principles of Automated Theorem Proving*. John Wiley & Sons, Inc., New York, NY.
- EIRÍKSSON, A. T. 1996. Integrating formal verification methods with a conventional project design flow. In *Proceedings of the 33rd Conference on Design Automation (DAC '96, Las Vegas, NV, June)*. 666–671.
- EIRÍKSSON, A. T. AND McMILLAN, K. L. 1995. Using formal verification/analysis methods on the critical path in system design. In *Proceedings of the Seventh International Conference on Computer-Aided Verification (CAV '95, July)*. Lecture Notes in Computer Science, vol. 939. Springer-Verlag, New York, 367–380.
- EMERSON, E. A. 1990. Temporal and modal logic. In *Handbook of Theoretical Computer Science (vol. B): Formal Models and Semantics*, J. van Leeuwen, Ed. MIT Press, Cambridge, MA, 995–1072.
- EMERSON, E. A. AND HALPERN, J. Y. 1986. “Sometimes” and “not never” revisited: On branching versus linear time temporal logic. *J. ACM* 33, 1 (Jan. 1986), 151–178.
- ENGBERG, U., GRÖNNING, P., AND LAMPORT, L. 1992. Mechanical verification of concurrent systems with TLA. In *Proceedings of the Fourth International Conference on Computer-Aided Verification (CAV '92)*. Lecture Notes in Computer Science, vol. 663. Springer-Verlag, New York, 44–55.
- FILKORN, T. 1991. A method for symbolic verification of synchronous circuits. In *Computer Hardware Description Languages and their Applications (CHDL '91, Apr.)*, D. Borrione and R. Waxman, Eds. North-Holland Publishing Co., Amsterdam, The Netherlands, 249–259.

- FUJITA, M., FUJISAWA, H., AND MATSUNAGA, Y. 1993. Variable ordering algorithms for ordered binary decision diagrams and their evaluation. *IEEE Trans. Comput.-Aided Des. Integr. Circuits* 12, 1 (Jan.), 6–12.
- GARLAND, S. J. AND GUTTAG, J. V. 1989. An overview of LP, the Larch prover. In *Rewriting Techniques and Applications*. Lecture Notes in Computer Science, vol. 355. Springer-Verlag, New York, 137–151.
- GERTH, R. 1989. Foundations of compositional program refinement: Safety properties. In *Stepwise Refinement of Distributed Systems*. Lecture Notes in Computer Science, vol. 430. Springer-Verlag, New York, 777–808.
- GORDON, M. 1985. Why higher-order logic is a good formalism for specifying and verifying hardware. In *Formal Aspects of VLSI Design*, G. J. Milne and P. A. Subrahmanyam, Eds. Elsevier Science Inc., New York, NY, 153–177.
- GORDON, M. J., Ed. 1988. HOL: A proof generating system for higher-order logic. In *VLSI Specification, Verification and Synthesis*, G. Birtwistle and P. Subrahmanyam, Eds. Kluwer Academic Publishers, Hingham, MA, 74–128.
- GORDON, M. J. C., WADSWORTH, C. P., AND MILNER, A. J. 1979. Edinburgh LCF: a mechanised logic of computation. In *Lecture Notes in Computer Science*. Lecture Notes in Computer Science, vol. 78. Springer-Verlag, New York.
- GRUMBERG, O. AND LONG, D. E. 1994. Model checking and modular verification. *ACM Trans. Program. Lang. Syst.* 16, 3 (May 1994), 843–871.
- GUPTA, A. 1992. Formal hardware verification methods: A survey. *Formal Methods Syst. Des.* 1, 2/3 (Oct. 1992), 151–238.
- HARDIN, R. H., HAR'EL, Z., AND KURSHAN, R. P. 1996. COSPAN. In *Proceedings of the Eighth International Conference on Computer-Aided Verification (CAV '96, Aug.)*. Lecture Notes in Computer Science, vol. 1102. Springer-Verlag, New York, 423–427.
- HAREL, D. 1988. On visual formalisms. *Commun. ACM* 31, 5 (May 1988), 514–530.
- HARKNESS, C. AND WOLF, E. 1994. Verifying the Summit bus converter protocols with symbolic model checking. *Formal Methods Syst. Des.* 4, 2 (Feb. 1994), 83–97.
- HARRISON, J. 1993. A HOL decision procedure for elementary real algebra. In *HUG '93*. Lecture Notes in Computer Science, vol. 780. Springer-Verlag, New York, 426–436.
- HARRISON, J. 1995. Floating point verification in HOL. In *Proceedings of the Eighth International Workshop on Higher Order Logic Theorem Proving and Its Applications (HOL '95)*. Lecture Notes in Computer Science, vol. 971. Springer-Verlag, New York, 186–199.
- HAZELHURST, S. 1996. Compositional model checking of partially ordered state spaces. Ph.D. Dissertation. University of British Columbia, Vancouver, Canada.
- HAZELHURST, S. AND SEGER, C.-J. H. 1994. Composing symbolic trajectory evaluation results. In *Proceedings of the Sixth International Conference on Computer-Aided Verification (CAV'94, June)*. Lecture Notes in Computer Science, vol. 818. Springer-Verlag, New York, 273–285.
- HAZELHURST, S. AND SEGER, C.-J. H. 1995. A simple theorem prover based on symbolic trajectory evaluation and BDDs. *IEEE Trans. Comput.-Aided Des. Integr. Circuits* 14, 4 (Apr.), 413–422.
- HENNESSY, J. L. AND PATTERSON, D. A. 1990. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann Publishers Inc., San Francisco, CA.
- HOARE, C. 1969. An axiomatic basis for computer programming. *Commun. ACM* 12, 10, 576–583.
- HOARE, C. 1978. Communicating sequential processes. *Commun. ACM* 21, 8, 666–677.
- HOJATI, R., BRAYTON, R. K., AND KURSHAN, R. P. 1993. BDD-based debugging of designs using language containment and fair CTL. In *Proceedings of the Fifth International Conference on Computer-Aided Verification (CAV '93, June)*. Lecture Notes in Computer Science, vol. 697. Springer-Verlag, New York, 41–58.
- HOJATI, R., SINGHAL, V., AND BRAYTON, R. K. 1994. Edge-Streett/Edge-Rabin automata environment for formal verification using language containment. Memo. UCB/ERL M94/12. Electronics Research Lab., UC Berkeley, Berkeley, CA.
- HOLZMANN, G. J. 1997. The model checker SPIN. *IEEE Trans. Softw. Eng.* 23, 5 (May), 279–295.

- HU, A. J. AND DILL, D. L. 1993. Reducing BDD size by exploiting functional dependencies. In *Proceedings of the 30th International Conference on Design Automation (DAC'93)*, Dallas, TX, June 14–18), A. E. Dunlop, Ed. ACM Press, New York, NY, 266–271.
- HU, A. J., DILL, D. L., DREXLER, A. J., AND YANG, C. H. 1992. Higher-level specification and verification with BDDs. In *Proceedings of the Fourth International Conference on Computer-Aided Verification (CAV '92)*. Lecture Notes in Computer Science, vol. 663. Springer-Verlag, New York, 82–95.
- HU, A. J., YORK, G., AND DILL, D. L. 1994. New techniques for efficient verification with implicitly conjoined BDDs. In *Proceedings of the 31st Annual Conference on Design Automation (DAC'94)*, San Diego, CA, June 6–10, 1994), M. Lorenzetti, Ed. ACM Press, New York, NY, 276–282.
- HUNT, W. A. 1989. Microprocessor design verification. *J. Autom. Reasoning* 5, 4 (Dec. 1989), 429–460.
- HUNT, W. A. 1994. FM8501: A verified microprocessor. In *Lecture Notes in Computer Science*, vol. 795. Springer-Verlag, New York.
- IP, C. N. AND DILL, D. L. 1996. Better verification through symmetry. *Formal Methods Syst. Des.* 9, 1/2 (Aug.), 41–75.
- IP, C. N. AND DILL, D. L. 1996. State reduction using reversible rules. In *Proceedings of the 33rd Conference on Design Automation (DAC '96)*, Las Vegas, NV, June). 564–567.
- JACKSON, P. B. 1992. Nuprl and its use in circuit design. In *Proceedings of the First International Conference on Theorem Provers in Circuit Design (TPCD '92)*, V. Stavridou, T. Melham, and R. Boute, Eds. North-Holland Publishing Co., Amsterdam, The Netherlands.
- JAIN, J., ABRAHAM, J. A., BITNER, J., AND FUSSELL, D. S. 1996. Probabilistic verification of boolean functions. *Formal Methods Syst. Des.* 1, 1 (July), 63–115.
- JAIN, J., NARAYAN, A., AND FUJITA, M. 1997. Formal verification of combinational circuits. In *Proceedings of the Tenth International Conference on VLSI Design*. 218–225.
- JONES, R. B., DILL, D. L., AND BURCH, J. R. 1995. Efficient validity checking for processor verification. In *Proceedings of the 1995 IEEE/ACM International Conference on Computer-Aided Design (ICCAD-95)*, San Jose, CA, Nov. 5–9), R. Rudell, Ed. IEEE Computer Society Press, Los Alamitos, CA, 2–6.
- JOYCE, J. J. 1988. Formal verification and implementation of a microprocessor. In *VLSI Specification, Verification and Synthesis*, G. Birtwistle and P. Subrahmanyam, Eds. Kluwer Academic Publishers, Hingham, MA, 129–157.
- JOYCE, J. J. AND SEGER, C.-J. H. 1993. Linking BDD-based symbolic evaluation to interactive theorem-proving. In *Proceedings of the 30th International Conference on Design Automation (DAC'93)*, Dallas, TX, June 14–18), A. E. Dunlop, Ed. ACM Press, New York, NY, 469–474.
- KAPUR, D. AND ZHANG, H. 1988. RRL: A rewrite rule laboratory. In *Proceedings of the Ninth International Conference on Automated Deduction (CADE '88)*. Lecture Notes in Computer Science, vol. 310. Springer-Verlag, New York, 768–769.
- KAUFMANN, M. AND MOORE, J. S. 1994. Design goals for ACL2. Tech. Rep. 101. Computational Logic, Inc., Austin, TX.
- KAUFMANN, M. AND MOORE, J. S. 1996. ACL2: an industrial strength version of Nqthm. In *Proceedings of the 11th Annual Conference on Computer Assurance (COMPASS '96)*, Gaithersburg, MD, June), S. Faulk and C. Heitmayer, Eds. 23–34.
- KESTEN, Y., MANNA, Z., MCGUIRE, H., AND PNUELI, A. 1993. A decision algorithm for full propositional temporal logic. In *Proceedings of the Fifth International Conference on Computer-Aided Verification (CAV '93)*, June). Lecture Notes in Computer Science, vol. 697. Springer-Verlag, New York, 97–109.
- KLEENE, S. C. 1967. *Mathematical Logic*. John Wiley & Sons, Inc., New York, NY.
- KLOOS, C. D. AND BREUER, P. 1995. *Formal Semantics for VHDL*. Kluwer Academic Publishers, Hingham, MA.
- KOZEN, D. 1993. Results on the propositional μ -calculus. *Theor. Comput. Sci.* 27, 3 (Dec.), 333–354.
- KUEHLMANN, A., SRINIVASAN, A., AND LAPOTIN, D. P. 1995. Verity—a formal verification program for custom CMOS circuits. *IBM J. Res. Dev.* 39, 1/2 (Jan./Mar. 1995), 149–165.

- KUMAR, R., BLUMENRÖHR, C., EISENBIEGLER, D., AND SCHMID, D. 1996. Formal synthesis in circuit design -- a classification and survey. In *Proceedings of the First International Conference on Formal Methods in Computer-Aided Design (FMCAD '96)*. Lecture Notes in Computer Science, vol. 1166. Springer-Verlag, New York, 294–309.
- KUMAR, R., SCHNEIDER, K., AND KROPF, T. 1993. Structuring and automating hardware proofs in a higher-order theorem-proving environment. *Formal Methods Syst. Des.* 2, 2 (Apr. 1993), 165–223.
- KURSHAN, R. P. 1989. Analysis of discrete event coordination. In *Stepwise Refinement of Distributed Systems*. Lecture Notes in Computer Science, vol. 430. Springer-Verlag, New York, 414–453.
- KURSHAN, R. P. 1997. Formal verification in a commercial setting. In *Proceedings of the 34th Conference on Design Automation (DAC '97, June)*.
- KURSHAN, R. P. AND LAMPORT, L. 1993. Verification of a multiplier: 64 bits and beyond. In *Proceedings of the Fifth International Conference on Computer-Aided Verification (CAV '93, June)*. Lecture Notes in Computer Science, vol. 697. Springer-Verlag, New York, 166–179.
- LAI, Y.-T. AND SASTRY, S. 1992. Edge-valued binary decision diagrams for multi-level hierarchical verification. In *Proceedings of the 29th ACM/IEEE Conference on Design Automation (DAC '92, Anaheim, CA, June 8-12)*, D. G. Schweikert, Ed. IEEE Computer Society Press, Los Alamitos, CA, 608–613.
- LAMPORT, L. 1980. “Sometime” is sometimes “not never”- on the temporal logic of programs. In *Proceedings of the Seventh Annual ACM Symposium on Principles of Programming Languages*. ACM Press, New York, NY, 174–185.
- LAMPORT, L. AND SCHNEIDER, F. B. 1984. The “Hoare logic” of CSP, and all that. *ACM Trans. Program. Lang. Syst.* 6, 2 (Apr. 1984), 281–296.
- LEE, T. W., GREENSTREET, M. R., AND SEGER, C.-J. 1994. Automatic verification of asynchronous circuits. *IEEE Des. Test* 12, 1, 24–31.
- LEE, T. W., GREENSTREET, M. R., AND SEGER, C. -J. 1994. Automatic verification of refinement. In *Proceedings of the IEEE International Conference on Computer Design (Cambridge, MA, Oct. 10-12)*.
- LEESER, M. AND O'LEARY, J. 1995. Verification of a subtractive radix-2 square root algorithm and implementation. In *Proceedings of the International Conference on Computer Design (ICCD '95, Oct.)*. 526–531.
- LENOSKI, D., LAUDON, J., GHARACHORLOO, K., WEBER, W.-D., GUPTA, A., HENNESSY, J., HOROWITZ, M., AND LAM, M. S. 1992. The Stanford Dash multiprocessor. *IEEE Computer* 25, 3 (Mar.), 63–79.
- LESLIE, I. AND MCAULEY, D. 1991. Fairisle: An ATM network for the local area. *SIGCOMM Comput. Commun. Rev.* 21, 4 (Sept. 1991), 327–336.
- LEVITT, J. AND OLUKOTUN, K. 1996. A scalable formal verification methodology for pipelined microprocessors. In *Proceedings of the 33rd Conference on Design Automation (DAC '96, Las Vegas, NV, June)*. 558–563.
- MADRE, J.-C. AND BILLON, J.-P. 1988. Proving circuit correctness using formal comparison between expected and extracted behaviour. In *Proceedings of the 25 Conference on Design Automation (DAC'88)*. 205–210.
- MADRE, J. C., COUDERT, O., AND BILLON, J. P. 1989. Automating the diagnosis and the rectification of design errors with PRIAM. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD, Nov.)*. 30–33.
- MALIK, S., WANG, A., BRAYTON, R., AND SANGIOVANNI-VINCENTELLI, A. 1988. Logic verification using binary decision diagrams in a logic synthesis environment. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD '88, Santa Clara, CA, Nov.)*. 6–9.
- MANNA, Z., ANUCHITANUKUL, A., BJØRNER, N., CHANG, E., COLÓN, M., DE ALFARO, L., DEVARAJAN, H., SIPMA, H., AND URIBE, T. 1994. STeP: The Stanford temporal prover. Tech. Rep. STAN-CS-TR-94-1518. Computer Systems Laboratory, Stanford Univ., Stanford, CA.
- MANNA, Z. AND PNUELI, A. 1992. *The Temporal Logic of Reactive and Concurrent Systems*. Springer-Verlag, New York, NY.

- MANNA, Z. AND WOLPER, P. 1984. Synthesis of communicating processes from temporal logic specifications. *ACM Trans. Program. Lang. Syst.* 6, 1 (Jan.), 68–93.
- MARETTI, N. 1994. Mechanized verification of refinement. In *Proceedings of the Second International Conference on Theorem Provers in Circuit Design (TPCD'94)*. Lecture Notes in Computer Science, vol. 901. Springer-Verlag, New York, 185–202.
- MCCUNE, W. 1994. OTTER 3.0. Preprint MCS-P399-1193. Argonne National Laboratory, Argonne, IL.
- McFARLAND, M. C. 1993. Formal verification of sequential hardware: A tutorial. *IEEE Trans. Comput.-Aided Des. Integr. Circuits* 12, 5 (May), 663–654.
- McMILLAN, K. L. 1992. Symbolic model checking—an approach to the state explosion problem. Ph.D. Dissertation. School of Computer Science, Carnegie Mellon University, Pittsburgh, PA.
- McMILLAN, K. L. 1994. Fitting formal methods into the design cycle. In *Proceedings of the 31st Annual Conference on Design Automation (DAC'94, San Diego, CA, June 6–10, 1994)*, M. Lorenzetti, Ed. ACM Press, New York, NY, 314–319.
- McMILLAN, K. L. 1996. A conjunctively decomposed boolean representation for symbolic model checking. In *Proceedings of the Eighth International Conference on Computer-Aided Verification (CAV '96, Aug.)*. Lecture Notes in Computer Science, vol. 1102. Springer-Verlag, New York, 13–25.
- McMILLAN, K. L. AND SCHWALBE, J. 1991. Formal verification of the Encore Gigamax cache consistency protocol. In *Proceedings of the 1991 International Symposium on Shared Memory Multiprocessors*.
- MELHAM, T. F. 1988. Abstraction mechanisms for hardware verification. In *VLSI Specification, Verification and Synthesis*, G. Birtwistle and P. Subrahmanyam, Eds. Kluwer Academic Publishers, Hingham, MA, 267–291.
- MILLER, S. P. AND SRIVAS, M. 1995. Formal verification of the AAMP5 microprocessor: A case study in the industrial use of formal methods. In *Proceedings of the Workshop on Industrial-Strength Formal Specification Techniques (WIFT '95) (WIFT '95, Boca Raton, FL)*. IEEE Computer Society, Washington, DC, 2–16.
- MINER, P. S. AND LEATHRUM, J. F. JR. 1996. Verification of IEEE compliant subtractive division algorithms. In *Proceedings of the First International Conference on Formal Methods in Computer-Aided Design (FMCAD '96)*. Lecture Notes in Computer Science, vol. 1166. Springer-Verlag, New York, 64–78.
- MOORE, J. S., LYNCH, T., AND KAUFMANN, M. 1996. Mechanically checked proof of the correctness of the kernel of the AMD_K86™ floating-point division algorithm. <http://devil.ece.utexas.edu:80/~lynch/divide/divide.html>.
- NELSON, G. AND OPPEN, D. C. 1979. Simplification by cooperating decision procedures. *ACM Trans. Program. Lang. Syst.* 1, 2 (Oct.), 245–257.
- NOWATZYK, A., AYBAY, G., BROWNE, M., KELLY, E., PARKIN, M., RADKE, W., AND VISHIN, S. 1995. The S3.mp scalable shared memory multiprocessor. In *Proceedings of the 9th International Symposium on Parallel Processing* (Apr.).
- OWICKI, S. AND LAMPORT, L. 1982. Proving liveness properties of concurrent programs. *ACM Trans. Program. Lang. Syst.* 4, 3 (July), 455–495.
- OWRE, S., RAJAN, S., RUSHBY, J., SHANKAR, N., AND SRIVAS, M. 1996. PVS: Combining specification, proof checking, and model checking. In *Proceedings of the Eighth International Conference on Computer-Aided Verification (CAV '96, Aug.)*. Lecture Notes in Computer Science, vol. 1102. Springer-Verlag, New York.
- OWRE, S., RUSHBY, J. M., AND SHANKAR, N. 1992. PVS: A prototype verification system. In *Proceedings of the 11th Conference on Automated Deduction (CADE'92)*. Lecture Notes in Computer Science, vol. 607. Springer-Verlag, New York, 748–752.
- PANDA, S. AND SOMENZI, F. 1995. Who are the variables in your neighborhood. In *Proceedings of the 1995 IEEE/ACM International Conference on Computer-Aided Design (ICCAD-95, San Jose, CA, Nov. 5–9)*, R. Rudell, Ed. IEEE Computer Society Press, Los Alamitos, CA, 74–77.

- PANDEY, M. AND RAIMI, R. 1996. Formal verification of PowerPC™ arrays using symbolic trajectory evaluation. In *Proceedings of the 33rd Conference on Design Automation (DAC '96, Las Vegas, NV, June)*. 649–654.
- PAULSON, L. C. 1994. Isabelle: A generic theorem prover. In *Lecture Notes in Computer Science*, vol. 828. Springer-Verlag, New York.
- PIXLEY, C., JEONG, S.-W., AND HACHTEL, G. D. 1994. Exact calculation of synchronizing sequences based on binary decision diagrams. *IEEE Trans. Comput.-Aided Des. Integr. Circuits* 13, 8 (Aug.), 1024–1034.
- PNUELI, A. 1977. The temporal logic of programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*. IEEE Computer Society Press, Los Alamitos, CA, 46–57.
- PNUELI, A. 1986. Applications of temporal logic to the specification and verification of reactive systems: A survey of current trends. In *Current Trends in Concurrency*, J. W. de Bakker, W.-P. de Roever, and G. Rozenberg, Eds. Lecture Notes in Computer Science, vol. 224. Springer-Verlag, New York, 510–584.
- PONG, F. AND DUBOIS, M. 1993. The verification of cache coherence protocols. In *Proceedings of the 5th Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA '93, Velen, Germany, June 30–July 2, 1993)*, L. Snyder, Ed. ACM Press, New York, NY, 11–20.
- PONG, F., NOWATZYK, A., AYBAY, G., AND DUBOIS, M. 1995. Verifying distributed directory-based cache coherence protocols: S3.mp, a case study. In *Proceedings of the EURO-Par '95 Conference on Parallel Processing*. Lecture Notes in Computer Science, vol. 966. Springer-Verlag, New York, 207–300.
- RAJAN, S., SHANKAR, N., AND SRIVAS, M. 1995. An integration of model-checking with automated proof checking. In *Proceedings of the Seventh International Conference on Computer-Aided Verification (CAV '95, July)*. Lecture Notes in Computer Science, vol. 939. Springer-Verlag, New York, 84–97.
- RAVI, K. AND SOMENZI, F. 1995. High-density reachability analysis. In *Proceedings of the 1995 IEEE/ACM International Conference on Computer-Aided Design (ICCAD-95, San Jose, CA, Nov. 5–9)*, R. Rudell, Ed. IEEE Computer Society Press, Los Alamitos, CA, 154–158.
- RUDELL, R. 1993. Dynamic variable ordering for ordered binary decision diagrams. In *Proceedings of the International Conference on Computer-Aided Design (ICCAD '93, Nov. 1993)*. 42–47.
- RUSS, H., SHANKAR, N., AND SRIVAS, N. M. 1996. Modular verification of SRT division. In *Proceedings of the Eighth International Conference on Computer-Aided Verification (CAV '96, Aug.)*. Lecture Notes in Computer Science, vol. 1102. Springer-Verlag, New York.
- SAXE, J. B., HORNING, J. J., GUTTAG, J. V., AND GARLAND, S. J. 1993. Using transformations and verification in circuit design. *Formal Methods Syst. Des.* 3, 3 (Dec. 1993), 181–209.
- SCHNEIDER, K. AND KROPP, T. 1996. A unified approach for combining different formalisms for hardware verification. In *Proceedings of the First International Conference on Formal Methods in Computer-Aided Design (FMCAD '96)*. Lecture Notes in Computer Science, vol. 1166. Springer-Verlag, New York, 202–217.
- SEGER, C.-J. H. 1993. Voss: A formal hardware verification system, user's guide. Tech. Rep. TR-93-45. University of British Columbia, Vancouver, Canada.
- SEGER, C.-J. H. AND BRYANT, R. E. 1995. Formal verification by symbolic evaluation of partially-ordered trajectories. *Formal Methods Syst. Des.* 6, 2 (Mar. 1995), 147–189.
- SENTOVICH, E., SINGH, K., LAVAGNO, L., MOON, C., MURGAI, R., SALDANHA, A., SAVOJ, H., STEPHAN, P., BRAYTON, R., AND SANGIOVANNI-VINCENTELLI, A. 1992. SIS: A system for sequential circuit synthesis. Tech. Rep. UCB/ERL M92/41. UC Berkeley, Berkeley, CA.
- SHANKAR, A. U. 1993. An introduction to assertional reasoning for concurrent systems. *ACM Comput. Surv.* 25, 3 (Sept. 1993), 225–262.
- SHOSTAK, R. E. 1979. A practical decision procedure for arithmetic with function symbols. *J. ACM* 26, 2 (Apr.), 351–360.
- SHOSTAK, R. E. 1984. Deciding combinations of theories. *J. ACM* 31, 1 (Jan. 1984), 1–12.
- SISTLA, A. P. AND CLARKE, E. M. 1985. The complexity of propositional linear temporal logics. *J. ACM* 32, 3 (July 1985), 733–749.

- SRIVAS, M. AND BICKFORD, M. 1990. Formal verification of a pipelined microprocessor. *IEEE Softw.* 7, 5 (Sept.), 52–64.
- SRIVAS, M. K. AND MILLER, S. P. 1996. Applying formal verification to the AAMP5 microprocessor: A case study in the industrial use of formal methods. *Formal Methods Syst. Des.* 8, 2, 153–188.
- STAUNSTRUP, J. 1994. *A Formal Approach to Hardware Design*. Kluwer Academic Publishers, Hingham, MA.
- STAUNSTRUP, J., GARLAND, S. J., AND GUTTAG, J. V. 1992. Mechanized verification of circuit descriptions using the Larch prover. In *Proceedings of the First International Conference on Theorem Provers in Circuit Design* (TPCD '92), V. Stavridou, T. Melham, and R. Boute, Eds. North-Holland Publishing Co., Amsterdam, The Netherlands.
- STAUNSTRUP, J. AND MELLERGAARD, N. 1995. Localized verification of modular designs. *Formal Methods Syst. Des.* 6, 3 (June 1995), 295–320.
- STERN, U. AND DILL, D. L. 1995. Automatic verification of the SCI cache coherence protocol. In *Correct Hardware Design and Verification Methods* (CHARME '95, Oct.). Lecture Notes in Computer Science, vol. 987. Springer-Verlag, New York, 21–34.
- TAHAR, S. AND CURZON, P. 1996. A comparison of MDG and HOL for hardware verification. In *Theorem Proving in Higher Order Logics*. Lecture Notes in Computer Science, vol. 1125. Springer-Verlag, New York, 415–430.
- TAHAR, S. AND KUMAR, R. 1993. Implementing a methodology for formally verifying RISC processors in HOL. In *HUG '93*. Lecture Notes in Computer Science, vol. 780. Springer-Verlag, New York, 281–294.
- TAHAR, S., ZHOU, Z., SONG, X., CERNY, E., AND LANGEVIN, M. 1996. Formal verification of an ATM switch fabric using Multiway Decision Graphs. In *Proceedings of the Great Lakes Symposium on VLSI* (Ames, IA). IEEE Computer Society Press, Los Alamitos, CA.
- TARSKI, A. 1951. *Decision Method for Elementary Algebra and Geometry*. University of California Press, Berkeley, CA.
- THOMAS, W. 1990. Automata on infinite objects. In *Handbook of Theoretical Computer Science (vol. B): Formal Models and Semantics*, J. van Leeuwen, Ed. MIT Press, Cambridge, MA, 133–191.
- TOUATI, H. J., SAVOJ, H., LIN, B., BRAYTON, R. K., AND SANGIOVANNI-VINCENTELLI, A. 1990. Implicit state enumeration of finite state machines using BDDs. In *Proceedings of the International Conference on Computer-Aided Design* (ICCAD'90). 130–133.
- VAN TASSEL, J. P. 1992. A formalism of the VHDL simulation cycle. In *Proceedings of the International Workshop on Higher Order Logic Theorem Proving and Its Applications*, L. J. M. Claesen and M. J. C. Gordon, Eds. Elsevier North-Holland, Inc., Amsterdam, The Netherlands, 359–374.
- VARDI, M. AND WOLPER, P. 1986. An automata-theoretic approach to automatic program verification. In *Proceedings of the First Annual Symposium on Logic in Computer Science* (Cambridge, MA, June 16–18). IEEE Computer Society Press, Los Alamitos, CA, 332–344.
- VARDI, M. AND WOLPER, P. 1986. Automata-theoretic techniques for modal logics of programs. *J. Comput. Syst. Sci.* 32, 2 (Apr. 1986), 183–221.
- VARDI, M. Y. AND WOLPER, P. 1994. Reasoning about infinite computations. *Inf. Comput.* 115, 1 (Nov. 15, 1994), 1–37.
- VETTER, R. J. 1995. ATM concepts, architectures, and protocols. *Commun. ACM* 38, 2 (Feb. 1995), 30ff.
- WEIH, D. AND GREENSTREET, M. 1996. Verifying asynchronous data path circuits. *IEE Proc. Comput. Digit. Tech.* 143, 5 (Sept.), 295–300.
- WINDLEY, P. J. 1992. Abstract theories in HOL. In *Proceedings of the International Workshop on Higher Order Logic Theorem Proving and Its Applications*, L. J. M. Claesen and M. J. C. Gordon, Eds. Elsevier North-Holland, Inc., Amsterdam, The Netherlands.
- WINDLEY, P. J. 1995. Formal modeling and verification of microprocessors. *IEEE Trans. Comput.* 44, 1 (Jan. 1995), 54–72.
- WINDLEY, P. J. 1995. Verifying pipelined microprocessors. Tech. Rep.. Brigham Young University, Provo, UT.

- WINDLEY, P. J. AND COE, M. 1994. A correctness model for pipelined microprocessors. In *Proceedings of the Second International Conference on Theorem Provers in Circuit Design (TPCD'94)*. Lecture Notes in Computer Science, vol. 901. Springer-Verlag, New York, 33–51.
- WOLPER, P. 1984. Temporal logic can be more expressive. *Inf. Control* 56, 1/2 (Jan./Feb. 1983), 72–99.
- XU, Y., CERNY, E., SILBURT, A., AND HUGHES, R. B. 1997. Property verification using theorem proving and model checking.
- ZHU, Z. AND SEGER, C.-J. H. 1994. The completeness of a hardware inference system. In *Proceedings of the Sixth International Conference on Computer-Aided Verification (CAV'94, June)*. Lecture Notes in Computer Science, vol. 818. Springer-Verlag, New York, 286–298.

Received: March 1996; revised: October 1997; revised: January 1998;